

# Ion.

The complete guide to building AI agents in plain English.

```
agent buddy:
    model "gemini/gemini-2.5-flash"
    role "You are a friendly assistant."

say ask buddy "Hello there!"
```

Version 1.0 · October 13, 2026

# Contents


Chapter 1 **Welcome and setup**

What is Ion? · How this book works · What you need · Installing · Your first program · The four commands you will use · Exercises

Chapter 2 **The shape of an Ion program**

Programs run top to bottom · Comments · Blocks and indentation · Names · Line breaks · The say statement · Variables with set · Putting values inside text · Trying things live: the REPL · Exercises

Chapter 3 **Values: text, numbers, and more**

Text · Numbers · Yes and no · Nothing · Truthiness: what counts as "yes" · Converting between types · Exercises

Chapter 4 **Operators and expressions**

Arithmetic · Comparing values · Looking inside things · And, or, not · Operator order (precedence) · Adding lists · Exercises

Chapter 5 **Making decisions**

if · otherwise · Combining conditions · Checking for empty things · Checking for nothing · Exercises

Chapter 6 **Repeating things**

repeat N times · for each · while · repeat forever and stop · skip · A pattern: summing and collecting · Exercises

Chapter 7 **Lists and maps**

Lists · Maps · JSON · Exercises

Chapter 8 **Functions**

Defining and calling · Several inputs, or none · Functions can call themselves · Where variables live (scope) · Functions are values · Describing a function · Exercises

Chapter 9 **When things go wrong**

Reading an error message · Catching errors with try · Stopping early · Checking before you run · Exercises

Chapter 10 **The built-in toolbox**

Text · Numbers · Lists and maps · Data · The outside world · Working with agents · Your own functions · Exercises

Chapter 11 **Meet agents**

Your first agent · The settings of an agent · The role · Memory · Abilities: what an agent is allowed to do · ask in detail · The chat statement · Names must be unique · Exercises

Chapter 12 **Models and API keys**

Choosing a model · Five ways to provide a key · Different keys for different agents · When the key is missing or wrong · Keeping costs under control · The mock and demo models · Exercises

Chapter 13 **Tools: giving agents new powers**

Your first tool · The describe line is how the AI understands · Inputs and results · When a tool fails · Tools are also ordinary functions · Many tools, many agents · Names · Exercises

Chapter 14 **Teams: agents working together**

Delegation: can ask · Teams with team · Three ways to organize work · What each agent sees · Limits that protect you · A review loop · Exercises

Chapter 15 **Talking to your agents: chat, serve and the API**

chat with · serve · Headless: use Ion as a back end · The simple HTTP API · Live updates with WebSocket · Exercises

Chapter 16 **The web UI**

A tour · Watching the team work · Permission cards · Rich answers · Addressing someone directly · The command menu · Keyboard shortcuts · Themes, phone, voice · History, export and starting over · Previewing without any setup · Limits

Chapter 17 **Splitting a program across files**

Chapter 18 **Command-line reference**

ion run · ion check · ion repl · ion preview · The .env file · Old-style files

Chapter 19 **Cookbook: complete programs**

A study buddy · Summarize a file · Turn messy text into data · An offline customer-support bot with a tool · Compare several AIs · A research team with a web page · Retry when a service is busy · Remember things between runs · Process a list of items · Using an agent inside your own function

Chapter 20 **Debugging and FAQ**

A checklist when something is wrong · Common mistakes · FAQ · Where to go from here

Appendix A **Cheat sheet**

Statements · Agents, tools and teams · Expressions · Commands

Appendix B **Reserved words**

Appendix C **Error message glossary**

Appendix D **Grammar summary**

Appendix E **The older ion init syntax**

Appendix F **Answers to the exercises**

Chapter 1 · Chapter 2 · Chapter 3 · Chapter 4 · Chapter 5 · Chapter 6 · Chapter 7 · Chapter 8 · Chapter 9 · Chapter 10 · Chapter 11 · Chapter 12 · Chapter 13 · Chapter 14

CHAPTER 1

# Welcome and setup

## 1.1 What is Ion?

Ion is a programming language built for one job: **making AI agents**. It reads almost like English, so you can describe what you want and let the AI do the thinking.

Here is a complete Ion program. Even if you have never programmed, you can probably guess what it does:

ION

```
agent buddy:
    model "mock"
    role "You are a friendly assistant."

say ask buddy "Hello there!"
```

It creates an agent called `buddy`, gives it a personality, asks it a question, and prints the answer. That is the heart of Ion. Everything else in this book is either:

- **ordinary programming**, such as variables, decisions, loops and functions, so you can control *what* you ask and *what you do with the answers*; or
- **agent features**, such as abilities, tools, teams, and a ready-made chat web page, so your agents can *do things* and *work together*.

## 1.2 How this book works

The book moves from simple to advanced. If you are new to programming, read Chapters 2 to 9 in order and type the examples yourself. If you already program, skim Chapters 2 to 10 and start properly at Chapter 11.

Throughout the book you will see these boxes:

> **TRY IT**
> Something to type and run yourself. Learning happens in your fingers.

> **TIP**
> A shortcut or a good habit.

> **WARNING**
> Something that can bite you, or cost you money, or run something on your computer.

> **NOTE**
> Extra detail you can skip on a first read.

Code samples that show their output were **actually run** when this book was built. The output you see is the output you will get. At the end of each chapter there are exercises, and **Appendix F has worked answers**.

## 1.3 What you need

- A computer with **Node.js 20 or newer** (check with `node --version`).
- The Ion project folder (the one containing this book).
- A text editor. Any editor works; Ion files end in `.ion`.
- *Later*, for real AI models: an API key from Google (Gemini), OpenAI, or Anthropic. **You do not need one for most of this book.** A built-in fake AI called `mock` lets you learn everything offline for free.

## 1.4 Installing

Open a terminal in the Ion folder and run:

```
TEXT

npm install
```

You can now run Ion with:

```
TEXT

npx tsx src/cli/index.ts run hello.ion
```

That is long to type. To get a short `ion` command, build once and link it:

```
TEXT

npm run build
npm link
```

After that you can type:

```
TEXT

ion run hello.ion
```

> **NOTE**
> This book writes commands as `ion run ...`. If you have not run `npm link`, type `npx tsx src/cli/index.ts` wherever you see `ion`.

## 1.5 Your first program

Create a file called `hello.ion` with this content:

```ion
say "Hello, world!"
```

```
Hello, world!
```

Run it with `ion run hello.ion`. That is a complete program: one line, one `say`.

## 1.6 The four commands you will use

| Command | What it does |
|---|---|
| `ion run file.ion` | Runs your program. |
| `ion check file.ion` | Finds mistakes **without running anything** (free, instant, no AI calls). |
| `ion repl` | An interactive prompt: type a line, see the result immediately. |
| `ion preview` | Opens the chat web page with a built-in demo team. No key needed. |

Chapter 18 covers every option.

> **TRY IT**
> Run `ion preview` now. A chat page opens. Type `work`, then `permission`, then `markdown`. This is the interface your own agents will get automatically. Press Ctrl+C in the terminal to stop it.

## 1.7 Exercises

1. Make `hello.ion` print your own name.
2. Add a second `say` line. In what order do the lines print?
3. Run `ion repl`, type `say "hi"`, and press Enter. Then type `exit`.

CHAPTER 2

# The shape of an Ion program

## 2.1 Programs run top to bottom

An Ion program is a list of **statements**, one per line. Ion runs them in order, from the first line to the last.

ION

```
say "first"
say "second"
say "third"
```

OUTPUT

```
first
second
third
```

## 2.2 Comments

A `#` starts a comment. Ion ignores everything from the `#` to the end of the line. Use comments to explain *why* you did something.

ION

```
# This whole line is a comment
say "hello"   # and this part is too
```

OUTPUT

```
hello
```

## 2.3 Blocks and indentation

Some statements contain other statements. The container line **ends with a colon**, and the lines inside it are **indented** (moved right) by four spaces:

ION

```
repeat 2 times:
    say "inside the block"
    say "still inside"
say "outside again"
```

OUTPUT

```
inside the block
still inside
inside the block
still inside
outside again
```

The rules are simple:

- The line before a block ends with `:`.
- Every line in the block is indented the **same amount**. Four spaces is the standard.
- A line that is indented *less* ends the block.
- Use spaces, not tabs. (A tab counts as four spaces, but mixing them causes confusion.)

> **TIP**
> If Ion says "This line is indented differently from the lines above it", some line in a block has a different number of spaces. Align them.

## 2.4 Names

Variables, functions, agents and tools all have **names**. A name:

- starts with a letter or an underscore;
- continues with letters, digits, or underscores;
- is case sensitive, so `Name` and `name` are different;
- cannot be one of Ion's **reserved words** (listed in Appendix B), such as `say`, `if`, `agent`.

Good names describe the thing: `total_price`, `research_agent`, `user_name`.

## 2.5 Line breaks

Normally one statement is one line. But inside brackets `( )`, `[ ]` and `{ }`, Ion ignores line breaks, so you can spread long lists out:

ION

```ion
set colors to [
    "red",
    "green",
    "blue"
]
say length(colors)
```

OUTPUT

```
3
```

## 2.6 The `say` statement

`say` prints a value. It shows text without quotes, numbers without extra zeros, and so on:

ION

```ion
say "words"
say 42
say 3.5
say yes
say [1, 2, 3]
```

OUTPUT

```
words
42
3.5
yes
[1, 2, 3]
```

## 2.7 Variables with `set`

A **variable** is a name that holds a value. Create or change one with `set ... to ...`:

ION

```ion
set name to "Ada"
set age to 36
say name
say age
set age to age + 1
say age
```

OUTPUT

```
Ada
36
37
```

The same `set` statement creates a variable the first time and changes it afterwards. There is no separate "declare" step.

> **NOTE**
> Ion uses the word `to`, never `=`. If you write `set x = 5`, Ion tells you to use `to`.

## 2.8 Putting values inside text

Put any expression inside `{ }` in a piece of text and Ion substitutes its value. This is called **interpolation**, and you will use it constantly, especially to build prompts for AI agents.

ION

```
set name to "Ada"
set age to 36
say "Hello, {name}! In 10 years you will be {age + 10}."
```

OUTPUT

```
Hello, Ada! In 10 years you will be 46.
```

## 2.9 Trying things live: the REPL

`ion repl` gives you a prompt. Type one statement and it runs at once. A bare expression prints its value, which makes the REPL a great calculator and a great way to learn:

TEXT

```
ion> set x to 4
ion> x * 2
8
ion> upper("hello")
HELLO
```

To type a block, end the first line with `:` and finish with an empty line.

## 2.10 Exercises

1. Create variables for your name and favorite number, then print a sentence using both.
2. What does `say 7 / 2` print? What about `say 7 % 2`? Find out in the REPL.
3. Why does this fail? Fix it.

ION

```
set count to 1
  say count
```

CHAPTER 3

# Values: text, numbers, and more

Every piece of data in Ion is a **value**, and every value has a **type**. There are six everyday types:

| Type | Examples | `type_of` says |
|---|---|---|
| Text | `"hello"`, `'hi'` | `text` |
| Number | `42`, `3.14`, `-7` | `number` |
| Yes/no | `yes`, `no`, `true`, `false` | `yes/no` |
| Nothing | `nothing` | `nothing` |
| List | `[1, 2, 3]` | `list` |
| Map | `{name: "Ada"}` | `map` |

Lists and maps get their own chapter (Chapter 7). This chapter covers the rest.

## 3.1 Text

Write text in double quotes or single quotes. They are identical, so pick whichever lets you include the other kind of quote easily:

ION

```
say "It's easy"
say 'She said "hi"'
```

OUTPUT

```
It's easy
She said "hi"
```

### Escapes

A backslash `\` inside text writes a special character:

| Write | Get |
| --- | --- |
| `\n` | a new line |
| `\t` | a tab |
| `\"` | a double quote |
| `\'` | a single quote |
| `\\` | a backslash |
| `\{` | a literal `{` |

ION

```
say "line one\nline two"
say "a\tb"
say "She said \"hi\""
```

OUTPUT

```
line one
line two
a	b
She said "hi"
```

## Interpolation

`{expression}` inside text is replaced by the expression's value. Any expression works: variables, arithmetic, function calls, even `ask`.

ION

```
set price to 4
say "{price} apples cost {price * 3} coins"
say "shouting: {upper("hi")}"
```

OUTPUT

```
4 apples cost 12 coins
shouting: HI
```

> **NOTE**
> A `{` only starts an interpolation when a **letter or underscore** follows it immediately. So JSON such as `{"a": 1}` or `{ 1 }` stays exactly as written. That matters when you ask AI to reply in JSON. To force a literal brace before a letter, write `\{`.

ION

```
```

```ion
say 'Reply like {"name": "Ada"}'
say "Literal: \{name}"
```

OUTPUT

```
Reply like {"name": "Ada"}
Literal: {name}
```

### Multi-line text

Three quotes in a row start a text that can span many lines. This is perfect for long AI instructions. Ion removes the shared indentation and the blank first and last lines:

ION

```ion
set topic to "cats"
say """
    You are a poet.
    Write about {topic}.
      (indented lines stay indented)
    """
```

OUTPUT

```
You are a poet.
Write about cats.
  (indented lines stay indented)
```

### Joining and measuring text

Join text with `+`. Anything added to text is converted to text first.

ION

```ion
say "Total: " + 5
say "a" + "b" + "c"
say length("hello")
say "hello"[0]
say upper("abc") + lower("DEF")
```

OUTPUT

```
Total: 5
abc
5
h
ABCdef
```

## 3.2 Numbers

Numbers are written the way you expect. There is one kind of number, with or without a decimal point. Negative numbers use a minus sign.

ION

```
say 10
say 2.5
say -3
say 0.1 + 0.2
say 1 / 4
```

OUTPUT

```
10
2.5
-3
0.3
0.25
```

Whole numbers print without `.0`, and long decimals are rounded to ten places, so `0.1 + 0.2` prints a tidy `0.3`.

Turn text into a number with `number(...)`. If the text is not a number, you get a clear error rather than a surprise:

ION

```
say number("42") + 1
say number(" 3.5 ") * 2
```

OUTPUT

```
43
7
```

## 3.3 Yes and no

The two yes/no values can be written `yes` and `no`, or `true` and `false`. They are interchangeable. Ion prints them as `yes` and `no`.

ION

```
set ready to yes
say ready
say 5 is greater than 3
say 5 is less than 3
```

OUTPUT

```
yes
yes
no
```

## 3.4 Nothing

`nothing` means "no value". Many things give you `nothing` when there is no answer: a missing map field, `first([])`, or `ask user` when input has ended.

ION

```ion
set person to {name: "Ada"}
say person.age
say first([])
say type_of(nothing)
```

OUTPUT

```
nothing
nothing
nothing
```

## 3.5 Truthiness: what counts as "yes"

Anywhere Ion needs a yes/no (in an `if`, a `while`, `and`, `or`, `not`), other values are converted. These are **false**:

- `no` / `false`
- `nothing`
- the number `0`
- empty text `""`
- an empty list `[]` and an empty map `{}`

**Everything else is true**, including the text `"no"` and the number `-1`. This lets you write natural checks:

ION

```ion
set name to ""
if name:
    say "have a name"
otherwise:
    say "no name yet"
if [1, 2]:
    say "list has items"
```

OUTPUT

```
no name yet
list has items
```

## 3.6 Converting between types

| Function | Does | Example |
|---|---|---|
| `text(x)` | any value to text | `text(5)` is `"5"` |
| `number(x)` | text to number | `number("7")` is `7` |
| `type_of(x)` | names the type | `type_of([])` is `"list"` |

ION

```ion
say type_of("hi")
say type_of(5)
say type_of(yes)
say type_of([1])
say type_of({a: 1})
say text(3) + text(4)
```

OUTPUT

```
text
number
yes/no
list
map
34
```

## 3.7 Exercises

1. Print a sentence that contains your name and your age inside one piece of text.
2. Write a multi-line text with three lines and print it.
3. What is `"5" + 3`? What is `5 + 3`? Why do they differ?
4. Which of these are false in an `if`: `0`, `"0"`, `[]`, `[0]`, `nothing`?

CHAPTER 4

# Operators and expressions

An **expression** is anything that produces a value: `5`, `name`, `price * 3`, `upper("hi")`. This chapter is about combining values.

## 4.1 Arithmetic

| Symbol | Words | Meaning |
|---|---|---|
| `+` | `plus` | add |
| `-` | `minus` | subtract |
| `*` | `times` | multiply |
| `/` | `divided by` | divide |
| `%` | `mod` | remainder |

Use whichever style you find clearer. They mean exactly the same thing.

ION

```
say 2 + 3 * 4
say (2 + 3) * 4
say 2 plus 3 times 4
say 10 divided by 4
say 10 minus 4
say 17 % 5
say 17 mod 5
```

OUTPUT

```
14
20
14
2.5
6
2
2
```

Multiplication and division happen before addition and subtraction, like in school maths. Use brackets to change the order.

Dividing by zero is an error, not a crash: `You can't divide by zero.`

## 4.2 Comparing values

Comparisons give a yes or no. Each has a plain-English form and a symbol form:

| English | Symbol |
|---|---|
| `is` | `==` |
| `is not` | `!=` |
| `is greater than` (or `more than`, `bigger than`) | `>` |
| `is less than` (or `fewer than`, `smaller than`) | `<` |
| `is at least` | `>=` |
| `is at most` | `<=` |

ION

```
set n to 5
say n is 5
say n is not 5
say n is greater than 3
say n is less than 3
say n is at least 5
say n is at most 4
say n == 5
say n >= 6
```

OUTPUT

```
yes
no
yes
no
yes
no
yes
no
```

Rules worth knowing:

- `is` compares by **value and type**. The text `"5"` is **not** the number `5`.
- Lists and maps are equal when all their contents are equal.
- `<`, `>`, `<=`, `>=` need two numbers or two pieces of text. Comparing a number with text is an error with a helpful message.

ION

```
say "5" is 5
say [1, 2] is [1, 2]
say {a: 1} is {a: 1}
say "apple" is less than "banana"
```

OUTPUT

```
no
yes
yes
yes
```

## 4.3 Looking inside things

| Form | Meaning |
|---|---|
| `x contains y` | text contains text; list has item; map has key |
| `x does not contain y` | the opposite |
| `y is in x` | same as `x contains y`, reading the other way |
| `x starts with y` | text starts with text |
| `x ends with y` | text ends with text |

ION

```
say "hello world" contains "world"
say [1, 2, 3] contains 2
say {name: "Ada"} contains "name"
say 4 is in [1, 2, 3]
say [1, 2] does not contain 9
say "report.pdf" ends with ".pdf"
say "Dr. Who" starts with "Dr"
```

OUTPUT

```
yes
yes
yes
no
yes
yes
yes
```

## 4.4 And, or, not

Combine yes/no values with `and`, `or`, `not`:

ION

```
set age to 30
set member to yes
say age is at least 18 and member
say age is less than 18 or member
say not member
```

OUTPUT

```
yes
yes
no
```

`and` and `or` stop early: `a and b` does not look at `b` if `a` is false. They also **return one of their inputs** rather than only yes or no, which gives you a neat way to supply a default:

ION

```
set nickname to ""
say nickname or "friend"
set other to "Sam"
say other or "friend"
```

OUTPUT

```
friend
Sam
```

## 4.5 Operator order (precedence)

From **tightest** to **loosest**:

| Level | Operators |
|---|---|
| 1 | calls `f(x)`, positions `x[0]`, fields `x.name` |
| 2 | negative sign `-x` |
| 3 | `* / % times divided by mod` |
| 4 | `+ - plus minus` |
| 5 | comparisons: `is`, `is not`, `<`, `contains`, `starts with` … |
| 6 | `not` |
| 7 | `and` |
| 8 | `or` |

When in doubt, add brackets. They cost nothing and make your intent obvious.

## 4.6 Adding lists

`+` joins two lists into a new one:

ION

```
say [1, 2] + [3]
```

OUTPUT

```
[1, 2, 3]
```

## 4.7 Exercises

1. Compute the average of 8, 12 and 22 in one expression.
2. Write an expression that is `yes` only if `age` is between 13 and 19, inclusive.
3. Given `set word to "Level"`, print whether `lower(word)` equals `"level"`.
4. What does `say 0 or "default"` print? Why?

CHAPTER 5

# Making decisions

## 5.1 `if`

An `if` runs its block only when the condition is true:

ION

```
set temperature to 31
if temperature is greater than 30:
    say "It's hot"
say "done"
```

OUTPUT

```
It's hot
done
```

## 5.2 `otherwise`

Add `otherwise:` for what to do when the condition is false. You can chain extra conditions with `otherwise if`. (The word `else` also works, if you prefer it.)

ION

```
set score to 85
if score is at least 90:
    say "A"
otherwise if score is at least 80:
    say "B"
otherwise if score is at least 70:
    say "C"
otherwise:
    say "Keep trying"
```

OUTPUT

```
B
```

Ion checks the conditions from top to bottom and runs the **first** one that is true, then skips the rest.

## 5.3 Combining conditions

Use `and`, `or` and `not` (Section 4.4):

ION

```
set age to 25
set has_ticket to yes
if age is at least 18 and has_ticket:
    say "Enter"
otherwise:
    say "Sorry"
```

OUTPUT

```
Enter
```

## 5.4 Checking for empty things

Because of truthiness (Section 3.5), you can test directly:

ION

```
set items to []
if not items:
    say "nothing in the list"
add "pen" to items
if items:
    say "now we have {length(items)}"
```

OUTPUT

```
nothing in the list
now we have 1
```

## 5.5 Checking for `nothing`

Compare with `nothing` using `is`:

ION

```
set person to {name: "Ada"}
if person.phone is nothing:
    say "no phone number"
```

OUTPUT

```
no phone number
```

## 5.6 Exercises

1. Print `"even"` or `"odd"` for a number `n` (hint: `n % 2`).
2. Convert a score to a letter grade A to F with `otherwise if`.
3. Print a greeting that depends on an `hour` variable: morning (before 12), afternoon (before 18), otherwise evening.

CHAPTER 6

# Repeating things

## 6.1 `repeat N times`

ION

```
repeat 3 times:
    say "hip hip hooray"
```

OUTPUT

```
hip hip hooray
hip hip hooray
hip hip hooray
```

## 6.2 `for each`

Go through the items of a list one at a time:

ION

```
for each fruit in ["apple", "pear", "plum"]:
    say "I like {fruit}"
```

OUTPUT

```
I like apple
I like pear
I like plum
```

`for each` also works on **text** (each character) and on **maps** (each key):

ION

```
for each letter in "abc":
    say letter
for each key in {x: 1, y: 2}:
    say key
```

OUTPUT

```
a
b
c
x
```

```
y
```

To count, use `range`. `range(5)` is `[0, 1, 2, 3, 4]` and `range(2, 5)` is `[2, 3, 4]`:

ION

```
for each i in range(1, 4):
    say "Step {i}"
```

OUTPUT

```
Step 1
Step 2
Step 3
```

## 6.3 `while`

Repeats as long as a condition stays true:

ION

```
set n to 1
while n is less than 100:
    set n to n * 2
say n
```

OUTPUT

```
128
```

> **WARNING**
> If the condition never becomes false, the loop never ends. Make sure something inside the loop changes it. Press **Ctrl+C** in the terminal to stop a runaway program.

## 6.4 `repeat forever` and `stop`

`stop` leaves the loop immediately. Use it with `repeat forever` when you do not know in advance how many rounds you need:

ION

```
set total to 0
set n to 1
repeat forever:
    set total to total + n
```

```
    if total is greater than 20:
        stop
    set n to n + 1
say "stopped at n={n}, total={total}"
```

OUTPUT

```
stopped at n=6, total=21
```

## 6.5 `skip`

`skip` jumps straight to the next round of the loop:

ION

```
for each n in range(1, 8):
    if n % 2 is 0:
        skip
    say n
```

OUTPUT

```
1
3
5
7
```

`stop` and `skip` only make sense inside a loop. Anywhere else, Ion tells you so.

## 6.6 A pattern: summing and collecting

Loops often build up a result. Start with an empty value, then update it each round:

ION

```
set squares to []
for each n in range(1, 6):
    add n * n to squares
say squares

set total to 0
for each s in squares:
    set total to total + s
say total
```

OUTPUT

```
[1, 4, 9, 16, 25]
55
```

## 6.7 Exercises

1. Print the numbers 1 to 10 with `for each` and `range`.
2. Print only the multiples of 3 up to 30.
3. Use `while` to find the first power of 2 above 1000.
4. Build a list of the first five even numbers.

CHAPTER 7

# Lists and maps

## 7.1 Lists

A **list** holds many values in order. Write it with square brackets:

ION

```
set fruits to ["apple", "pear", "plum"]
say fruits
say length(fruits)
say fruits.length
```

OUTPUT

```
["apple", "pear", "plum"]
3
3
```

A list can hold anything, including other lists:

ION

```
set grid to [[1, 2], [3, 4]]
say grid[1][0]
```

OUTPUT

```
3
```

### Positions start at 0

The first item is at position **0**, the second at 1, and so on. A negative position counts from the end, so `-1` is the last item.

ION

```
set xs to [10, 20, 30]
say xs[0]
say xs[2]
say xs[-1]
```

OUTPUT

```
10
30
30
```

Asking for a position that does not exist is an error that tells you how many items there are.

### Changing a list

ION

```
set xs to [10, 20]
add 30 to xs
say xs
remove 10 from xs
say xs
set xs[0] to 99
say xs
```

OUTPUT

```
[10, 20, 30]
[20, 30]
[99, 30]
```

- `add value to list` puts the value at the **end**.
- `remove value from list` removes the **first** matching item (nothing happens if there is none).
- `set list[position] to value` replaces an existing item.

### Slices, sorting and friends

ION

```
set xs to [5, 3, 9, 1]
say sort(xs)
say reverse(xs)
say first(xs)
say last(xs)
say slice(xs, 1, 3)
say unique([1, 1, 2, 2, 3])
say sum(xs)
say max(xs)
say min(xs)
```

OUTPUT

```
[1, 3, 5, 9]
[1, 9, 3, 5]
5
1
[3, 9]
```

```
[1, 2, 3]
18
9
1
```

`sort`, `reverse`, `slice` and `unique` **return a new list** and leave the original alone.

### Turning lists into text and back

ION

```ion
say join(["a", "b", "c"], "-")
say join([1, 2, 3])
say split("one two  three")
say split("a,b,c", ",")
```

OUTPUT

```
a-b-c
1, 2, 3
["one", "two", "three"]
["a", "b", "c"]
```

### Transforming lists with `map` and `filter`

Functions (Chapter 8) are values you can hand to other functions. `map` applies a function to every item and `filter` keeps the items for which the function says yes.

ION

```ion
to double(x):
    give back x * 2

to is_big(x):
    give back x is greater than 2

say map([1, 2, 3], double)
say filter([1, 2, 3, 4], is_big)
```

OUTPUT

```
[2, 4, 6]
[3, 4]
```

## 7.2 Maps

A **map** stores values under names (called **keys**), like a form with labelled fields:

ION

```
set person to {name: "Ada", age: 36}
say person.name
say person["age"]
say keys(person)
say values(person)
say length(person)
```

OUTPUT

```
Ada
36
["name", "age"]
["Ada", 36]
2
```

Keys that are not plain words can be written as text: `{"full name": "Ada Lovelace"}`, and read with `person["full name"]`.

### Changing a map

ION

```
set person to {name: "Ada"}
set person.age to 36
set person["city"] to "London"
say person
remove "city" from person
say person
say person contains "age"
```

OUTPUT

```
{"name":"Ada","age":36,"city":"London"}
{"name":"Ada","age":36}
yes
```

Reading a key that does not exist gives `nothing` (not an error), so you can check for it with `is nothing`.

### Maps inside lists inside maps

Real data is nested. Read it by chaining:

ION

```ion
set club to {
    name: "Rockets",
    members: [
        {name: "Ada", role: "lead"},
        {name: "Sam", role: "writer"}
    ]
}
say club.members[1].name
for each m in club.members:
    say "{m.name} is the {m.role}"
```

OUTPUT

```
Sam
Ada is the lead
Sam is the writer
```

## 7.3 JSON

AI models often answer in JSON, which is a common text format for data. Ion reads and writes it:

ION

```ion
set data to json('{"name": "Ada", "scores": [90, 85]}')
say data.name
say data.scores[1]
say to_json({city: "London", year: 1843})
```

OUTPUT

```
Ada
85
{"city":"London","year":1843}
```

`json(text)` is forgiving: if a model wraps its JSON in a code fence or adds a sentence before it, `json` still finds the data. `to_json(value, yes)` prints it indented.

## 7.4 Exercises

1. Make a list of five numbers and print their sum and the biggest one.
2. Make a map describing a book (title, author, year) and print a sentence from it.
3. Given `set words to ["pear", "fig", "apple"]`, print them sorted, then print only the words longer than 3 letters (hint: write a function and use `filter`).
4. Turn `{a: 1, b: 2}` into JSON text and back.

CHAPTER 8

# Functions

A **function** is a named, reusable piece of code. You write it once and use it many times.

## 8.1 Defining and calling

Start with `to`, then the name, then the inputs in brackets. Use `give back` to return a result:

ION

```
to greet(name):
    give back "Hello, " + name + "!"

say greet("Ada")
say greet("Sam")
```

OUTPUT

```
Hello, Ada!
Hello, Sam!
```

Call a function by writing its name and brackets. Always use brackets when calling, even with no inputs: `now()`.

## 8.2 Several inputs, or none

ION

```
to area(width, height):
    give back width * height

to hello():
    say "hi from a function"

say area(3, 4)
hello()
```

OUTPUT

```
12
hi from a function
```

Giving the wrong number of inputs is an error that spells out what the function expects. A function that never uses `give back` gives back `nothing`.

## 8.3 Functions can call themselves

This is called **recursion**. Every recursive function needs a way to stop:

ION

```
to factorial(n):
    if n is at most 1:
        give back 1
    give back n * factorial(n - 1)

say factorial(5)
```

OUTPUT

```
120
```

If a function keeps calling itself forever, Ion stops it with a clear message after 200 levels.

## 8.4 Where variables live (scope)

Inputs and variables created inside a function exist only inside it. But watch this rule:

> **`set` changes an existing variable if one with that name already exists *anywhere visible*. Otherwise it creates a new one.**

ION

```
set count to 10

to bump():
    set count to count + 1

bump()
bump()
say count
```

OUTPUT

```
12
```

The function changed the outer `count`, because one already existed. That is useful for counters, but a surprise if you did not intend it. Use distinctive names for temporary variables inside functions.

## 8.5 Functions are values

You can pass a function to `map` and `filter` by name (no brackets), and store it in a variable:

ION

```
to shout(text):
    give back upper(text) + "!"

say map(["hi", "bye"], shout)
set f to shout
say f("ok")
```

OUTPUT

```
["HI!", "BYE!"]
OK!
```

## 8.6 Describing a function

A function's first line may be a `describe` statement. It does nothing when the function runs, but it documents the function. For **tools** (Chapter 13) it is how you tell the AI what the tool is for.

ION

```
to celsius(f):
    describe "Converts Fahrenheit to Celsius"
    give back round((f - 32) * 5 / 9, 1)

say celsius(212)
say celsius(98.6)
```

OUTPUT

```
100
37
```

## 8.7 Exercises

1. Write `to double(n)` and use it on 21.
2. Write `to biggest(numbers)` that finds the largest number with a loop. Compare with `max`.
3. Write `to is_even(n)` that gives back yes or no, then use it with `filter` on `range(10)`.
4. Write `to repeat_text(text, count)` that gives back the text repeated `count` times.

CHAPTER 9

# When things go wrong

## 9.1 Reading an error message

Ion's errors are written for people. They tell you the file, the line, the problem, and often a fix:

ION

```
set total to 1
say totl
```

ERROR

```
script.ion:2: I don't know anything called 'totl'.
    say totl
  hint: Did you mean 'total'?
```

Always read the **line number** first, then the message, then the hint.

## 9.2 Catching errors with `try`

Some things fail for reasons you cannot control: a web page is down, an AI service is busy, a file is missing. Wrap the risky part in `try` and say what to do in `on error`:

ION

```
try:
    say 10 / 0
on error:
    say "Something went wrong: {error}"
say "the program keeps going"
```

OUTPUT

```
Something went wrong: You can't divide by zero.
the program keeps going
```

Inside `on error`, the variable `error` holds the message. Choose another name with `as`:

ION

```
try:
    say number("abc")
on error as problem:
    say "Bad number: {problem}"
```

```
OUTPUT
Bad number: I can't turn "abc" into a number.
```

`try` catches every runtime failure, including AI and network errors. Use it around anything that talks to the outside world (Chapter 20 shows a retry pattern).

## 9.3 Stopping early

An uncaught error stops the program and prints the message. The exit status of `ion run` is then non-zero, which is handy in scripts.

## 9.4 Checking before you run

`ion check file.ion` reads your program without running it and reports mistakes that do not need the program to run: unknown agents, tools, models, team members, misspelled settings. It is free and instant. Make a habit of running it.

## 9.5 Exercises

1. Write a program with a deliberate typo. Read the error. Then run `ion check` on it.
2. Wrap `number("12abc")` in `try` and print a friendly message.
3. Write a function `safe_divide(a, b)` that gives back `nothing` instead of failing when `b` is 0.

CHAPTER 10

# The built-in toolbox

These functions are always available. Where a function takes an optional input, it is shown in square brackets.

## 10.1 Text

| Function | Gives back | Example |
|---|---|---|
| `length(x)` | the size of text, a list or a map | `length("hello")` is `5` |
| `upper(t)` | UPPERCASE | `upper("hi")` is `"HI"` |
| `lower(t)` | lowercase | `lower("HI")` is `"hi"` |
| `trim(t)` | text without spaces at both ends | `trim(" a ")` is `"a"` |
| `split(t [, separator])` | a list of pieces; with no separator, splits on whitespace | `split("a b")` is `["a", "b"]` |
| `lines(t)` | a list of the lines in the text | `lines("a\nb")` is `["a", "b"]` |
| `join(list [, separator])` | one text; the separator defaults to `", "` | `join([1, 2], "-")` is `"1-2"` |
| `replace(t, find, with)` | text with every `find` replaced | `replace("aXa", "X", "-")` is `"a-a"` |
| `slice(t, start [, end])` | part of the text (end not included) | `slice("hello", 1, 3)` is `"el"` |
| `text(x)` | any value as text | `text(5)` is `"5"` |
| `number(x)` | text as a number (error if it is not one) | `number("4")` is `4` |
| `type_of(x)` | `text`, `number`, `yes/no`, `list`, `map`, `nothing`, `function`, `agent`, `team` | `type_of(1)` is `"number"` |

## 10.2 Numbers

| Function | Gives back | Example |
|---|---|---|
| `round(x [, digits])` | rounded number | `round(2.567, 2)` is `2.57` |
| `floor(x)` | round down | `floor(2.9)` is `2` |
| `ceil(x)` | round up | `ceil(2.1)` is `3` |
| `abs(x)` | without the minus sign | `abs(-4)` is `4` |
| `min(list)` or `min(a, b, ...)` | smallest number | `min(3, 1, 2)` is `1` |
| `max(list)` or `max(a, b, ...)` | largest number | `max([3, 9])` is `9` |
| `sum(list)` or `sum(a, b, ...)` | total | `sum([1, 2, 3])` is `6` |
| `random(low, high)` | a random whole number from low to high, **both included** | `random(1, 6)` |

## 10.3 Lists and maps

| Function | Gives back | Example |
|---|---|---|
| `range(n)` or `range(a, b)` | the list `0..n-1`, or `a..b-1` | `range(2, 5)` is `[2, 3, 4]` |
| `first(list)` / `last(list)` | the first or last item, or `nothing` if empty | `last([1, 2])` is `2` |
| `sort(list)` | a new sorted list | `sort([3, 1])` is `[1, 3]` |
| `reverse(list)` | a new reversed list | `reverse([1, 2])` is `[2, 1]` |
| `unique(list)` | a new list without repeats | `unique([1, 1])` is `[1]` |
| `slice(list, start [, end])` | part of the list | `slice([1, 2, 3], 1)` is `[2, 3]` |
| `keys(map)` / `values(map)` | the keys or the values as a list | `keys({a: 1})` is `["a"]` |
| `map(list, function)` | a new list with the function applied to each item | `map([1, 2], double)` |
| `filter(list, function)` | the items for which the function gives yes | `filter([1, 2], is_big)` |

## 10.4 Data

| Function | Gives back |
|---|---|
| `json(text)` | the data in the JSON text (forgives code fences and chatter around it) |
| `to_json(value [, pretty])` | JSON text. Pass `yes` as the second input for indented output |

## 10.5 The outside world

| Function | Does |
|---|---|
| `now()` | the current date and time as text (ISO format) |
| `today()` | today's date as text, like `2026-10-13` |
| `env(name)` | the value of an environment variable, or `nothing` |
| `sleep(seconds)` | waits |
| `read_file(path)` | the contents of a file as text |
| `write_file(path, text)` | writes a file (replacing it) |
| `fetch(url)` | the text of a web page or API response (an error if the request fails) |

> **WARNING**
> `read_file`, `write_file` and `fetch` are **your** program doing things, with no permission prompts. Permission prompts (Chapter 11) apply to what **agents** do. Your own script is trusted.

## 10.6 Working with agents

| Function | Does |
|---|---|
| `ask_all(list_of_agents, prompt)` | asks every agent in the list the same thing **at the same time** and gives back a list of replies in the same order |

Chapter 14 shows it in action.

## 10.7 Your own functions

Anything you define with `to` joins the toolbox. You can even replace a built-in name with your own function; yours wins.

## 10.8 Exercises

1. Use `split`, `map` and `join` to turn `"red green blue"` into `"RED, GREEN, BLUE"`.

2. Use `random` to simulate rolling two dice and print the total.
3. Use `today()` in a sentence: "Today is ...".

CHAPTER 11

# Meet agents

An **agent** is an AI with a name, a personality, and (optionally) abilities. You create one with `agent`, then talk to it with `ask`.

## 11.1 Your first agent

We use `model "mock"` for now. `mock` is a pretend AI built into Ion that needs no key and costs nothing. It simply repeats what you say. That is perfect for learning how everything fits together.

ION

```
agent buddy:
    model "mock"
    role "You are a friendly assistant."

set answer to ask buddy "Hello there"
say answer
```

OUTPUT

```
mock reply: Hello there
```

Two new things:

- The `agent` block defines the agent. Its settings are indented lines.
- `ask buddy "..."` sends a message and **gives back the reply as text**. Here we store it in `answer` and print it.

You can print directly too: `say ask buddy "Hello"`.

> **NOTE**
> With a real model, `buddy` would answer like a person: "*Hi there! How can I help?*" Chapter 12 shows how to switch. Everything else stays identical.

## 11.2 The settings of an agent

Each setting is one line inside the agent block:

| Setting | Meaning | Example |
|---|---|---|
| `model` | **Required.** Which AI to use. | `model "gemini/gemini-2.5-flash"` |
| `role` | The agent's instructions and personality. (`instructions` also works.) | `role "You are a pirate."` |
| `key` | An API key for the model (Chapter 12). | `key from env "MY_KEY"` |
| `can` | An ability the agent has (Section 11.5). | `can browse the web` |
| `remembers` / `forgets` | Whether it remembers earlier messages. | `forgets between asks` |
| `input` / `output` | Which modes the chat page offers. | `input text, voice` |

Only `model` is required.

## 11.3 The `role`

The role is the most powerful setting. It is the standing instruction the AI follows in every reply. Be specific. Say who the agent is, what it should do, and how it should answer.

For long instructions, use multi-line text (Section 3.1). You can even put variables inside:

ION

```
set audience to "ten year olds"
agent teacher:
    model "mock"
    role """
        You are a patient science teacher.
        Explain things so {audience} understand.
        Use one short example. Never use jargon.
        """

say ask teacher "why is the sky blue?"
```

OUTPUT

```
mock reply: why is the sky blue?
```

> **TIP**
> Treat the role like a job description for a new employee. Vague roles give vague answers.

## 11.4 Memory

By default an agent **remembers the conversation**. Each `ask` sees everything that came before. That is what makes follow-up questions work: "What is the capital of France?" then "How many people live there?"

ION

```
agent guide:
    model "gemini/gemini-2.5-flash"

say ask guide "What is the capital of France?"
say ask guide "How many people live there?"
```

The second question works because the agent remembers the first. If you want every `ask` to start fresh, add:

ION

```
agent translator:
    model "gemini/gemini-2.5-flash"
    forgets between asks
```

Use `forgets` for one-off jobs (translating, classifying, extracting data), where history would only add cost and confusion.

## 11.5 Abilities: what an agent is allowed to do

By default an agent can only talk. To let it act on your computer or the internet, you grant **abilities** with `can`:

| Line | What the agent gets |
|---|---|
| `can access files in "folder"` | Read, write and list files, **only inside that folder** |
| `can browse the web` | Fetch web pages, and open a real browser window |
| `can run commands` | Run commands in your terminal |
| `can read gmail with token from env "NAME"` | Search and read your Gmail |
| `can use tool_a, tool_b` | Use tools you wrote (Chapter 13) |
| `can ask agent_a, agent_b` | Hand tasks to other agents (Chapter 14) |

ION

```
agent librarian:
    model "gemini/gemini-2.5-flash"
    role "You help me find things in my notes."
    can access files in "./notes"
```

Now `ask librarian "What notes do I have about gardening?"` lets the agent list and read files in `./notes` to find the answer.

### How safe is this?

Abilities are **opt-in** and **narrow**:

- The agent gets only what you listed. Nothing else.
- `can access files in "./notes"` is a **fence**. A request to read outside the folder (for example `../secrets.txt`) is refused.
- **The first time** each agent uses each kind of ability in a run, Ion **asks you** (`Allow 'librarian' to use file-access (read 'a.txt')? [y/N]`). In the chat web page this is a card with Allow and Deny buttons. After you approve, that agent can keep using that ability for the rest of the run.
- `--yes` approves everything automatically. Only use it for programs you fully trust.

> **WARNING**
> `can run commands` is **not fenced**. The agent can run anything your own account can run. Grant it rarely, and read what the agent asks to run before pressing `y`.

> **WARNING**
> `can browse the web` includes `browse_page`, which opens a visible Chromium window. The first time, you may need to run `npx playwright install chromium`.

## 11.6 `ask` in detail

```
ION
set reply to ask agent_name "your message"
```

- The target is the name of an agent or team (or the word `user`).
- The message is any text expression, so `ask a "Summarize: " + article` works. (The message ends at a comparison or `and`; put brackets around anything fancier.)
- The result is **text**. If you asked for structured data, convert it with `json(...)`.
- If the AI service fails (bad key, no internet, rate limit), the error says so, and you can catch it with `try`.

### Asking the person at the keyboard

`ask user` prints your question and waits for a line of typing:

```ion
set name to ask user "What is your name?"
say "Nice to meet you, {name}!"
```

If input ends (for example the program is run with no keyboard attached), `ask user` gives back `nothing`.

## 11.7 The `chat` statement

For a quick terminal conversation with an agent, write `chat with`:

```ion
agent buddy:
    model "gemini/gemini-2.5-flash"
    role "You are a friendly assistant."

chat with buddy
```

Type messages and read answers. Type `exit`, `quit`, or `bye` to finish. A bare empty line is ignored.

## 11.8 Names must be unique

An agent, a team and a tool cannot share a name. If you try, Ion says `There is already an agent, team or tool called 'x'.`

## 11.9 Exercises

1. Create an agent with `model "mock"` and ask it two questions.
2. Create two agents and ask each one the same question. Print both replies.
3. Write a role that makes an agent answer only in rhymes. (You will see its effect once you use a real model.)
4. Decide: should a translator agent remember or forget? Why?

CHAPTER 12

# Models and API keys

## 12.1 Choosing a model

The `model` setting names the AI. Write it as `"provider/model-name"`:

```ion
agent a:
    model "gemini/gemini-2.5-flash"
```

| Provider | Write | Default model if you give only the provider | Key variable |
|---|---|---|---|
| Google Gemini | `gemini/...` | `gemini-2.5-flash` | `GEMINI_API_KEY` |
| OpenAI | `openai/...` | `gpt-4o-mini` | `OPENAI_API_KEY` |
| Anthropic | `anthropic/...` | `claude-sonnet-5` | `ANTHROPIC_API_KEY` |
| Mock (offline fake) | `mock` | n/a | none needed |
| Demo (scripted showcase) | `demo` | n/a | none needed |

The part after the slash is passed straight to the provider, so you can use **any model name your provider offers**. If you write only the provider (`model "openai"`), Ion uses the default from the table.

You can mix providers in one program. A common pattern is a cheap fast model for easy jobs and a stronger one for hard jobs.

Where to get a key:

- Gemini: Google AI Studio, `aistudio.google.com/apikey`
- OpenAI: `platform.openai.com/api-keys`
- Anthropic: `console.anthropic.com`

> **WARNING**
> API keys are like passwords **and like credit cards**. Anyone holding your key can spend your money. Never post one online, send it in a chat, or share a file that contains one.

## 12.2 Five ways to provide a key

Ion looks for a key in this order: a key written on the agent, then the environment variable. Pick the style that suits you.

### Way 1: an environment variable (recommended)

Set the variable with the default name from the table, and write nothing about keys in your program:

```text
PowerShell:   $env:GEMINI_API_KEY = "your-key"
Mac / Linux:  export GEMINI_API_KEY="your-key"
```

```ion
agent a:
    model "gemini/gemini-2.5-flash"
```

### Way 2: a `.env` file (easy and safe)

Create a file named `.env` next to your `.ion` file:

```text
GEMINI_API_KEY=your-key-here
OPENAI_API_KEY=your-other-key
```

Ion loads it automatically each time you run. Real environment variables win over the file. Lines starting with `#` are comments, and quotes around values are removed. Your program stays clean and shareable.

### Way 3: a different variable name

```ion
agent a:
    model "openai/gpt-4o-mini"
    key from env "WORK_OPENAI_KEY"
```

### Way 4: write the key right in the program (inline)

The quickest way, perfect for a quick experiment on your own machine:

```ion
agent buddy:
    model "gemini/gemini-2.5-flash"
    key "PASTE-YOUR-KEY-HERE"
```

`key is "..."` and `apikey "..."` mean the same thing, if you prefer how they read:

```ion
```

```
agent buddy:
    model "openai/gpt-4o-mini"
    apikey is "PASTE-YOUR-KEY-HERE"
```

> **WARNING**
> A key written in a program is **stored in plain text in that file**. If you email the file, post it, or upload it, you have shared your key. Use inline keys only for files that stay on your computer, and move to a `.env` file before you share anything. If a key ever leaks, delete it in your provider's dashboard and make a new one.

### Way 5: ask for it when the program starts

Nothing is stored anywhere. Ion asks you every run:

ION

```
set my_key to ask user "Paste your Gemini key:"

agent buddy:
    model "gemini/gemini-2.5-flash"
    key my_key
```

The `key` line accepts any text expression: a variable, `env("NAME")`, or a piece of text. (Typed characters are visible on screen, so use this where nobody is watching.)

## 12.3 Different keys for different agents

Each agent has its own `key`, so you can use separate accounts:

ION

```
agent cheap:
    model "openai/gpt-4o-mini"
    key from env "PERSONAL_OPENAI_KEY"

agent smart:
    model "anthropic/claude-sonnet-5"
    key from env "WORK_ANTHROPIC_KEY"
```

## 12.4 When the key is missing or wrong

If no key can be found, Ion says exactly what to do:

TEXT

```
Agent 'a' needs the environment variable GEMINI_API_KEY, but it isn't set.
  hint: Set it in your terminal, or put GEMINI_API_KEY=... in a file called .env next to your
script.
```

If a key is present but wrong, the AI provider rejects it and Ion shows its message after `Asking 'a' failed:`. Typical meanings:

| Message contains | Meaning |
|---|---|
| `401` or `403`, "API key not valid" | The key is wrong, expired or lacks permission |
| `429` | You are sending too fast, or you ran out of quota or credit |
| `404`, "model not found" | The model name after the slash is not one your account can use |
| `500`, `503` | The provider is having trouble. Try again shortly |

## 12.5 Keeping costs under control

- Every `ask` costs money with real models (cents or fractions of a cent for small models).
- Long memory means more text sent each time. Use `forgets between asks` for one-off jobs.
- An agent with tools may make several AI calls per `ask`. There is a safety cap of 25 rounds per message.
- Start with small cheap models while you develop. Switch to stronger ones when it works.
- Develop with `model "mock"` and switch the model line last. All your logic is tested for free.

## 12.6 The `mock` and `demo` models

`mock` repeats what you send, prefixed with `mock reply:`. It also has one trick: if your message is `CALL tool_name {"arg": "value"}`, it **calls that tool**, which lets you test tools and delegation without an AI. You will use this in the next two chapters.

`demo` powers `ion preview`. It reacts to a few keywords (`markdown`, `work`, `permission`, `research`) to show off the chat page.

## 12.7 Exercises

1. Create a `.env` file with a fake key, and run a program that uses a real model name. What error do you see? (A fake key is rejected by the provider, which is the correct and safe outcome.)
2. Write an agent that takes its key from a custom variable name.
3. Why is the `.env` approach better than an inline key when sharing code with a friend?

CHAPTER 13

# Tools: giving agents new powers

An agent that can only talk is limited. **Tools** are functions that **the agent can decide to call**. A tool can do anything you can program: look up a price, query a database, call a web service, calculate something exactly.

## 13.1 Your first tool

A tool looks like a function, but starts with `tool` and has a `describe` line:

ION

```
tool shout(text):
    describe "Makes text LOUD"
    give back upper(text) + "!"

agent echo:
    model "mock"
    can use shout

say ask echo 'CALL shout {"text": "tools give agents powers"}'
```

OUTPUT

```
Done: TOOLS GIVE AGENTS POWERS!
```

What happened:

1. `tool shout(text)` defined a tool with one input.
2. `can use shout` gave the agent permission to use it.
3. The mock AI saw `CALL shout {...}` and called the tool. (A **real** AI decides by itself when a tool would help, based on your `describe` text.)
4. The tool's result went back to the agent, which produced the final answer.

## 13.2 The `describe` line is how the AI understands

A real AI **reads the description** to decide whether and when to use the tool. Write it like instructions to a colleague. Mention what the tool does, what each input means, and what comes back.

ION

```
tool get_weather(city):
    describe "Gets the current weather for a city. Input: the city name, like 'Paris'. Returns a short summary."
    give back fetch("https://wttr.in/" + city + "?format=3")

agent assistant:
    model "gemini/gemini-2.5-flash"
    role "You help people plan their day."
    can use get_weather

say ask assistant "Should I take an umbrella in London today?"
```

Without a `describe` line the AI only sees the tool's name, so it will use it less wisely.

## 13.3 Inputs and results

- **Inputs** are named in the brackets: `tool book(room, day):`. The AI fills them in.
- **Results**: whatever the tool gives back. Text is handed over as is. Numbers, lists and maps are converted to JSON text for the AI.

ION

```
tool add_numbers(a, b):
    describe "Adds two numbers"
    give back a + b

agent calc:
    model "mock"
    can use add_numbers

say ask calc 'CALL add_numbers {"a": 2, "b": 3}'
```

OUTPUT

```
Done: 5
```

## 13.4 When a tool fails

If the tool hits an error (a web page is down, a bad input), Ion does **not** crash your program. The AI is told `Error: ...` and can try something else or explain the problem to the user.

## 13.5 Tools are also ordinary functions

You can call a tool yourself: `say shout("hello")` works. That makes tools easy to test before an AI ever uses them.

## 13.6 Many tools, many agents

Give an agent several tools, and share a tool between agents:

```ion
agent analyst:
    model "gemini/gemini-2.5-flash"
    can use get_weather, add_numbers, shout
```

> **WARNING**
> A tool runs **as your program**, with no permission prompt, because you wrote it and chose to give it to the agent. Think about what an AI could do with it. A tool that deletes files or sends money should check its inputs very carefully, or not exist.

## 13.7 Names

Tool names cannot be the same as the built-in agent tools: `read_file`, `write_file`, `list_files`, `fetch_url`, `browse_page`, `run_command`, `search_gmail`, `read_gmail_message`. Ion tells you if you collide.

## 13.8 Exercises

1. Write a tool `word_count(text)` that gives back the number of words, and test it by calling it directly.
2. Give it to a mock agent and trigger it with `CALL`.
3. Write a tool `tip(amount, percent)` that returns the tip, and call it from your script.
4. Think of a tool you would like an AI to have. What should its `describe` line say?

CHAPTER 14

# Teams: agents working together

One agent can do a lot. A **team** of specialists can do more: one researches, one writes, one checks the facts.

## 14.1 Delegation: `can ask`

An agent with `can ask other` can **hand tasks to another agent**. The AI decides when to do this.

ION

```
agent boss:
    model "mock"
    can ask worker

agent worker:
    model "mock"

say ask boss 'CALL delegate_to_worker {"task": "do the report"}'
```

OUTPUT

```
Done: mock reply: do the report
```

Behind the scenes, `can ask worker` gave `boss` a tool called `delegate_to_worker`. When the boss uses it, the `worker` receives the task as a message, replies, and the reply comes back to the boss as the tool result. (With a real AI, the boss decides when to delegate. Here the mock obeys `CALL`.)

## 14.2 Teams with `team`

A `team` bundles a **lead** and **members**:

ION

```
agent boss:
    model "mock"
agent writer:
    model "mock"
agent checker:
    model "mock"

team crew:
    lead boss
```

```
    members writer, checker

say ask crew 'CALL delegate_to_writer {"task": "write a haiku"}'
say ask crew 'CALL delegate_to_checker {"task": "check it"}'
```

OUTPUT

```
Done: mock reply: write a haiku
Done: mock reply: check it
```

`ask crew ...` talks to the **lead**, who can delegate to any member. The team is an agent-like value: you can `ask` it, `chat with` it, `serve` it, or list it in another agent's `can ask`.

The lead's own abilities and tools still apply, and the lead also tells its members apart by their abilities, so a lead with a good role (such as "Send research to the researcher, then ask the writer to save a report") will delegate sensibly.

## 14.3 Three ways to organize work

**1. Let the AI decide (a team).** Flexible. Good for open-ended requests. Less predictable.

ION

```
agent lead:
    model "gemini/gemini-2.5-flash"
    role "Send research questions to the researcher, then ask the writer to save a short
report."
agent researcher:
    model "gemini/gemini-2.5-flash"
    can browse the web
agent writer:
    model "gemini/gemini-2.5-flash"
    can access files in "./reports"

team crew:
    lead lead
    members researcher, writer
```

**2. You decide (a script).** Predictable and cheap. You control the order:

ION

```
set facts to ask researcher "Find three facts about volcanoes"
set report to ask writer "Write a friendly paragraph using these facts:\n{facts}"
say report
```

**3. In parallel.** Ask several agents at once with `ask_all`:

ION

```
agent optimist:
    model "mock"
agent skeptic:
    model "mock"

say ask_all([optimist, skeptic], "Is this plan good?")
```

OUTPUT

```
["mock reply: Is this plan good?", "mock reply: Is this plan good?"]
```

`ask_all` runs all the asks **at the same time**, so it takes as long as the slowest one rather than the sum. The replies come back as a list in the same order as the agents.

> **TIP**
> Prefer scripts when the steps are always the same. Prefer teams when the right steps depend on the request.

## 14.4 What each agent sees

- Every agent has **its own memory** of its own conversation.
- Delegated tasks arrive as messages to the specialist. They do **not** see the user's whole chat unless the lead includes it. So tell the lead (in its `role`) to **include the details** each specialist needs.
- Each agent uses its **own abilities**. If `researcher` browses the web, the permission card names the researcher.

## 14.5 Limits that protect you

- Agents asking each other can nest at most **six levels deep**. If your agents delegate in a circle, you get a clear error instead of an endless loop.
- Each message may involve at most **25** rounds of tool use per agent.
- An agent cannot ask itself, and `ion check` catches unknown names in `can ask` and `team` lines.

## 14.6 A review loop

A classic pattern: one agent writes, another critiques, repeat until approved:

ION

```
agent writer:
    model "gemini/gemini-2.5-flash"
    role "You write short product descriptions."
agent reviewer:
    model "gemini/gemini-2.5-flash"
    role "Reply APPROVED if the text is great. Otherwise give exactly one improvement."
    forgets between asks

set draft to ask writer "Write two sentences about a reusable water bottle."
repeat 3 times:
    set review to ask reviewer "Review this:\n{draft}"
    if review contains "APPROVED":
        stop
    set draft to ask writer "Improve it using this feedback: {review}"
say draft
```

## 14.7 Exercises

1. Build a team with a lead and two members, using `mock`. Trigger each member with `CALL`.
2. Write a script (not a team) that asks one mock agent, then passes its answer to another.
3. Use `ask_all` to ask three agents the same question and print each reply on its own line (hint: `for each`).

CHAPTER 15

# Talking to your agents: chat, serve and the API

You have three ways to let people (or other programs) talk to your agents.

| Way | Statement | Best for |
|---|---|---|
| Terminal chat | `chat with buddy` | Quick tests, one user |
| Web chat page | `serve buddy` | A real interface for people |
| API only | `serve buddy` with `--headless` | Connecting your own app or website |

## 15.1 `chat with`

ION

```
agent buddy:
    model "gemini/gemini-2.5-flash"
    role "You are a friendly assistant."

chat with buddy
```

Works for teams too: `chat with crew`.

## 15.2 `serve`

ION

```
serve crew
serve crew on port 5000
```

`serve` starts a small web server on your computer and keeps running until you press **Ctrl+C**. Without a port it uses **4310**. Your default browser opens the chat page automatically.

You choose who answers: `serve crew` means messages go to the team's lead (the **entry point**). Every other agent can still be addressed directly with `@name` in the chat.

The permission cards in the chat page are how you approve what agents try to do (Chapter 16). `ion run file.ion --yes` approves everything automatically.

> **NOTE**
>
> The server listens on **your computer only** (`127.0.0.1`), so other devices on your network cannot reach it. There is no login, and an agent may be able to read files or run commands, so keep it that way unless you add your own protection.

## 15.3 Headless: use Ion as a back end

```
TEXT

ion run crew.ion --headless
```

With `--headless` there is **no web page and no browser opens**. Only the API is available. This is how you put your agents behind your own website or app.

## 15.4 The simple HTTP API

Send a message and wait for the full answer:

```
TEXT

POST http://localhost:4310/message
Content-Type: application/json

{"text": "Hello!", "session_id": "alice"}
```

Reply:

```
TEXT

{"result": "Hi! How can I help?"}
```

From PowerShell:

```
TEXT

Invoke-RestMethod -Uri http://localhost:4310/message -Method Post -ContentType
"application/json" -Body '{"text":"Hello!","session_id":"alice"}'
```

From a shell with curl:

```
TEXT

curl -X POST http://localhost:4310/message -H "Content-Type: application/json" -d
'{"text":"Hello!","session_id":"alice"}'
```

- `session_id` is optional. Requests with the **same** session id share one conversation memory. With no id they all share the id `rest-default`.
- If something fails you get HTTP 500 and `{"error": "..."}`.
- A request over HTTP has no way to show a permission card, so **abilities that need permission are refused** unless you started Ion with `--yes`.

Two read-only endpoints describe your setup:

| Endpoint | Returns |
|---|---|
| `GET /config` | `{"entry": "boss", "input": ["text"], "output": ["text"]}` |
| `GET /agents` | a list of agents: `name`, `role`, `entry`, `team`, `isOrchestrator`, `grants`, `tools`, `teammates` |

## 15.5 Live updates with WebSocket

For a live experience (the chat page uses this), connect a WebSocket to `ws://localhost:4310/stream`. Each connection is one conversation.

You send JSON messages:

| You send | Meaning |
|---|---|
| `{"type": "message", "text": "Hi", "addressedTo": "helper"}` | Send a message. `addressedTo` is optional |
| `{"type": "confirm_response", "confirmId": "...", "ok": true}` | Answer a permission request |

You receive JSON events as the work happens:

| Event | Fields | Meaning |
|---|---|---|
| `chunk` | `agent`, `text` | An agent produced some text |
| `tool_call` | `agent`, `tool`, `detail` | An agent started using a tool. `delegate_to_x` means a hand-off to `x` |
| `tool_result` | `agent`, `tool`, `isError`, `ms`, `result` | The tool finished (result is shortened if very long) |
| `confirm_required` | `confirmId`, `agent`, `grant`, `detail` | Permission needed. Reply with `confirm_response` |
| `final` | `agent`, `text` | The finished answer. The turn is over |

A tiny web page using it:

```
TEXT
```

```
const ws = new WebSocket("ws://localhost:4310/stream");
ws.onopen = () => ws.send(JSON.stringify({ type: "message", text: "Hello" }));
ws.onmessage = (e) => {
  const event = JSON.parse(e.data);
  if (event.type === "final") console.log(event.text);
};
```

> **NOTE**
> A browser page served from another address cannot normally connect to this local server, and that is intentional. For your own site, put a small back end in front, or run Ion on the same address.

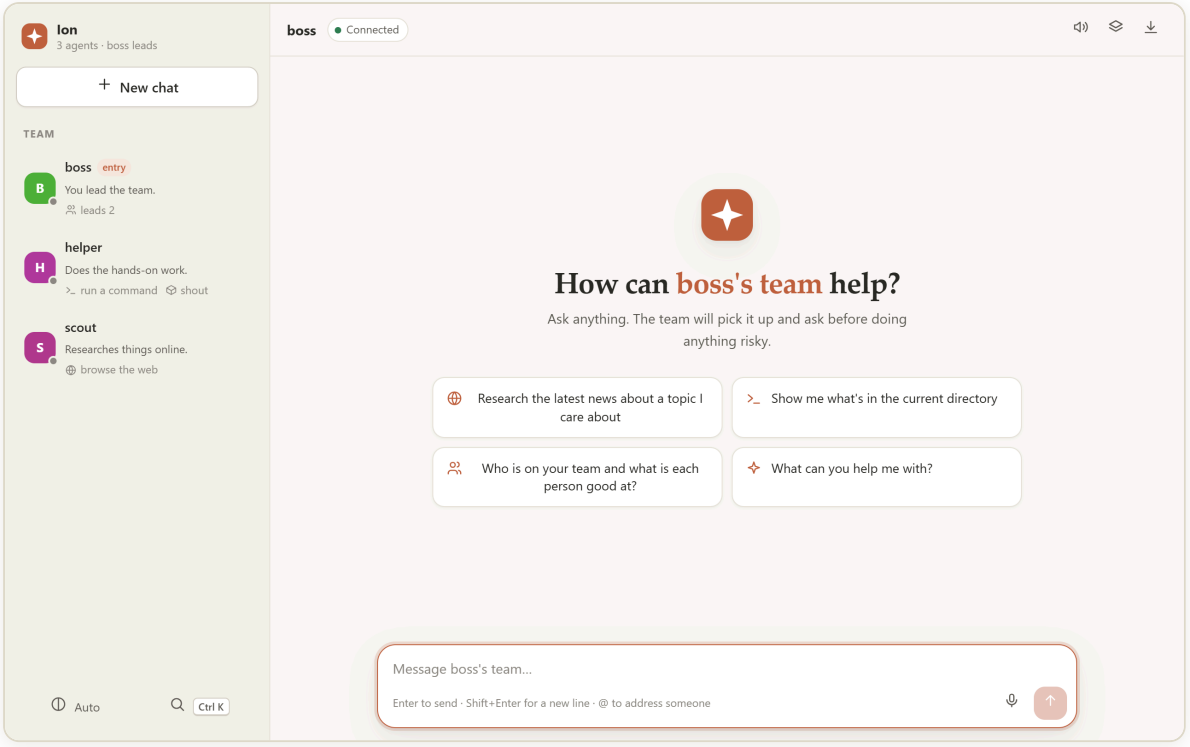
## 15.6 Exercises

1. Run the demo (`ion preview`) and use `GET /agents` in your browser: open `http://localhost:4310/agents`.
2. Start `crew.ion` with `--headless` and send it a message with PowerShell or curl.
3. Send two messages with the same `session_id` and check the agent remembers the first.

CHAPTER 16

# The web UI

`serve` (and `ion preview`) gives you a complete chat application with no setup.




The chat page with a team of three agents

## 16.1 A tour

**Left: the team.** Every agent has a colored avatar, its role, and its abilities (files, web, commands, tools). A green dot pulses when that agent is working. Click an agent to start a message addressed to it. Teams are labeled `team`, and the agent that answers by default is labeled `entry`.

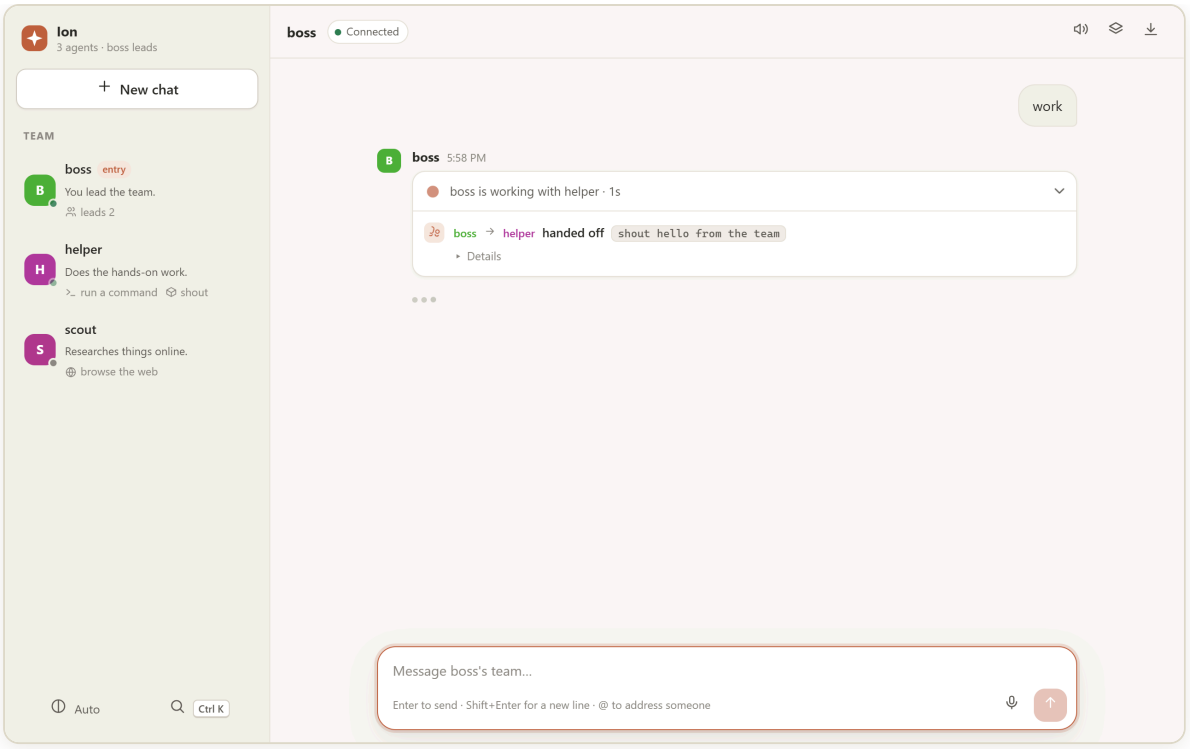
**Top: the title and connection light.** Green means connected. If the connection drops, Ion reconnects automatically and tells you the agents' short-term memory restarted.

**Center: the conversation.** Your messages are on the right. Agent replies are on the left, rendered as rich text.

**Bottom: the message box.** Press **Enter** to send and **Shift+Enter** for a new line. If you type while the team is busy, your message waits in a queue and sends when they are free.

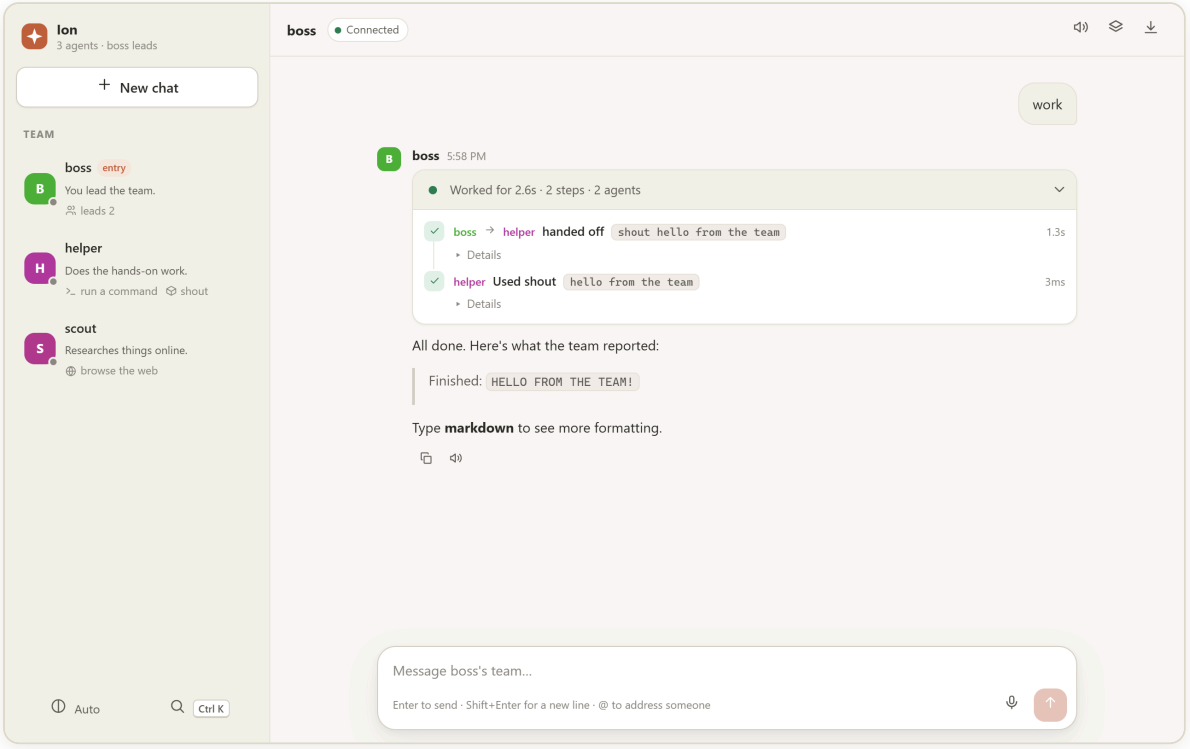
## 16.2 Watching the team work

While agents work, a live **activity** card shows each step: who handed work to whom, which tool is running, how long it took, and what came back.




A hand-off from boss to helper, live

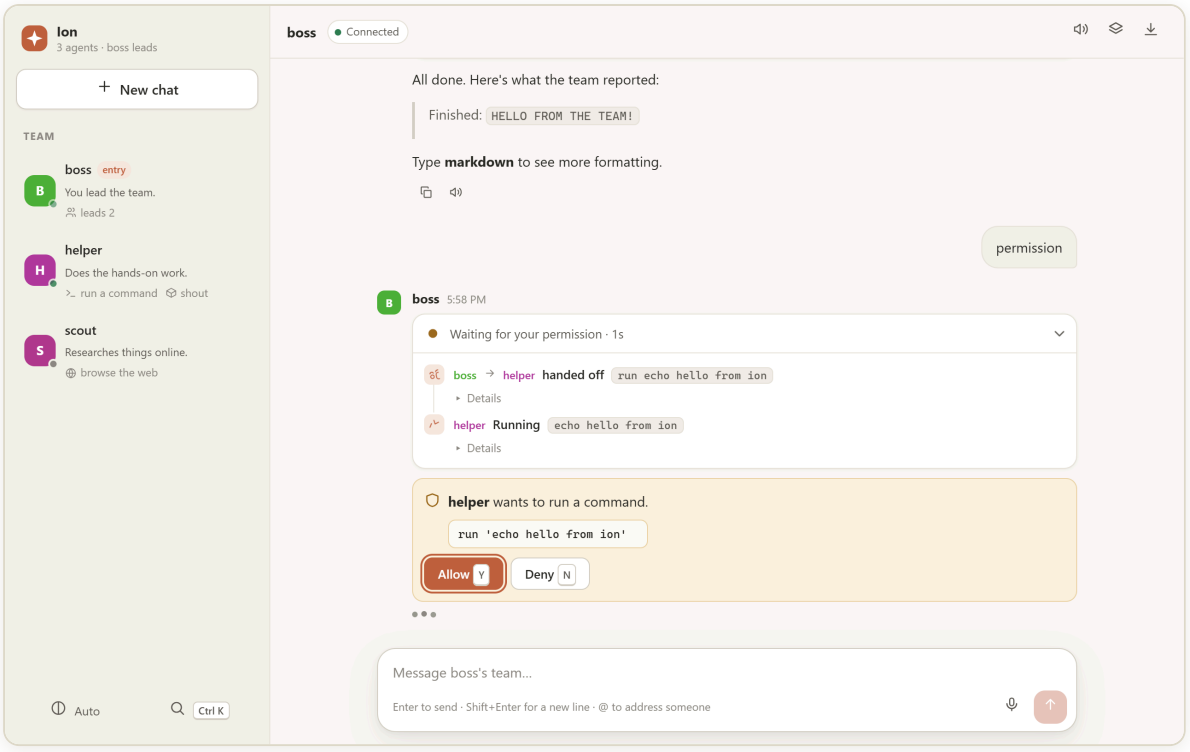
When the reply is ready, the card collapses to a summary like "Worked for 1.1s, 2 steps, 2 agents". Click it to expand the full timeline. Each step has a **Details** link showing the exact inputs and result.




The finished reply with its expanded timeline

## 16.3 Permission cards

When an agent wants to do something that needs permission, a card appears and the work **waits for you**:

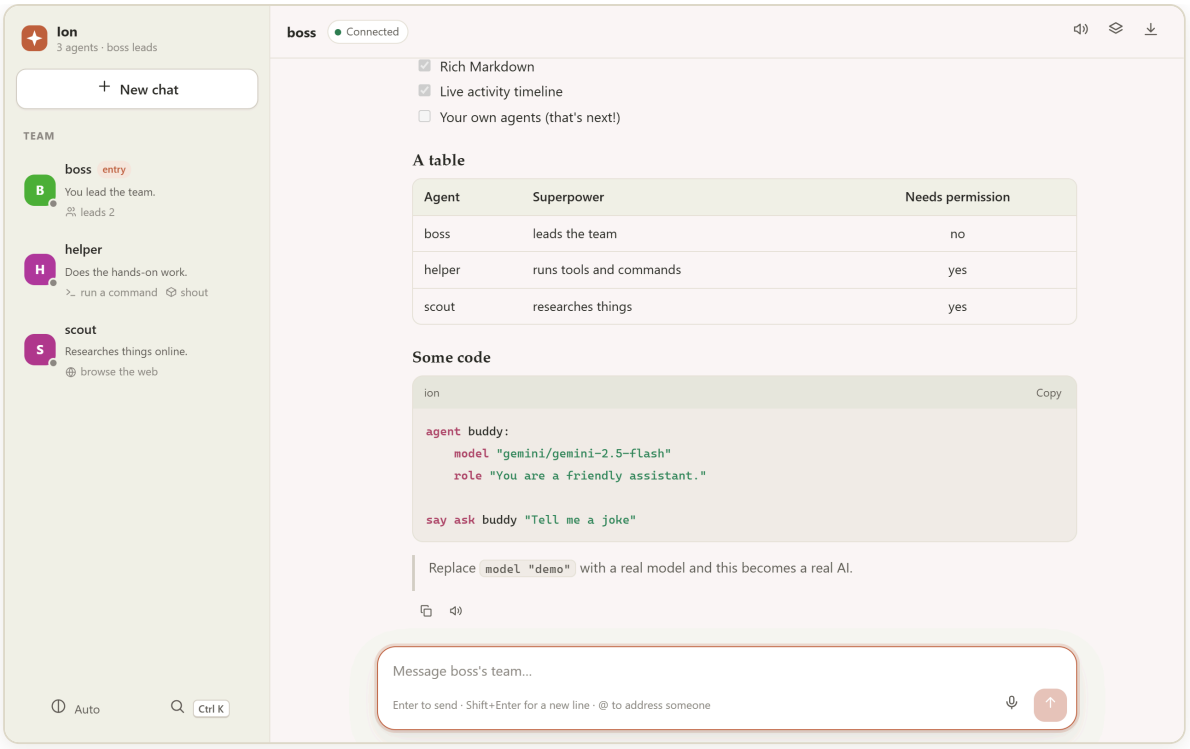



A permission request

Press **Y** to allow or **N** to deny, or click the buttons. If you switch to another tab, the page title shows a warning so you do not miss it. Once you allow a kind of ability for an agent, it is not asked again during that run.

## 16.4 Rich answers

Replies are rendered from Markdown: headings, bold and italic, lists, checklists, tables, quotes, links, and code with syntax highlighting and a **Copy** button.

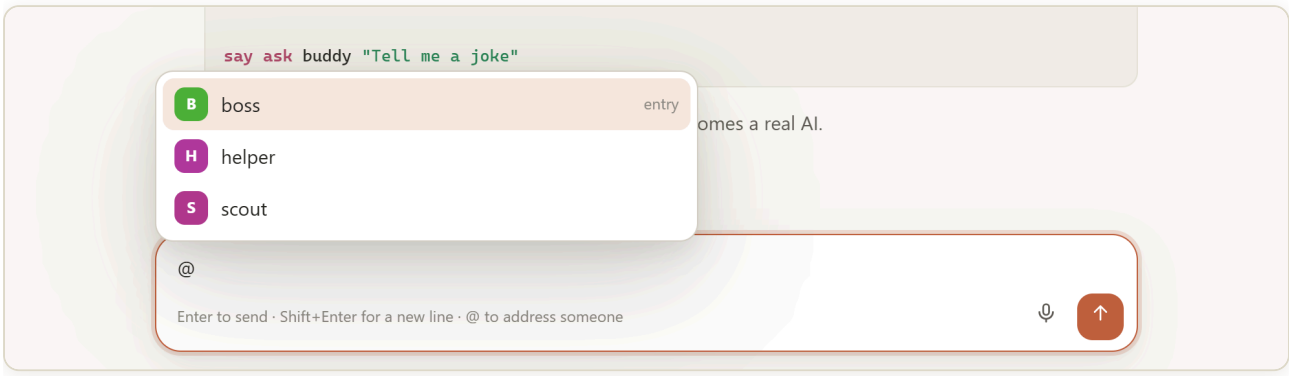



Markdown, a table, and highlighted code

Ask your agents to format answers with Markdown for the best results. Hover over a reply to **copy** it, **read it aloud**, or, if it failed, **try again**.

## 16.5 Addressing someone directly

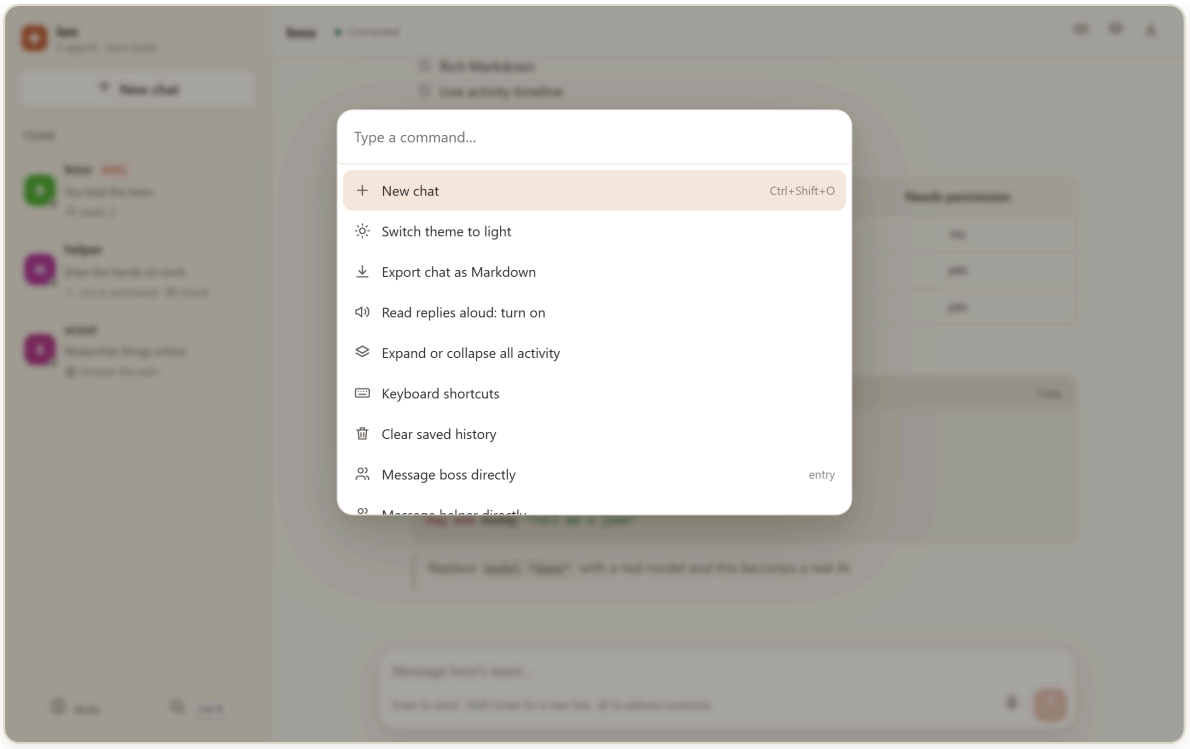
Type `@` at the start of a message to pick an agent, then press Tab or Enter:




The mention picker

## 16.6 The command menu

Press **Ctrl+K** (Cmd+K on a Mac) for a searchable list of actions:



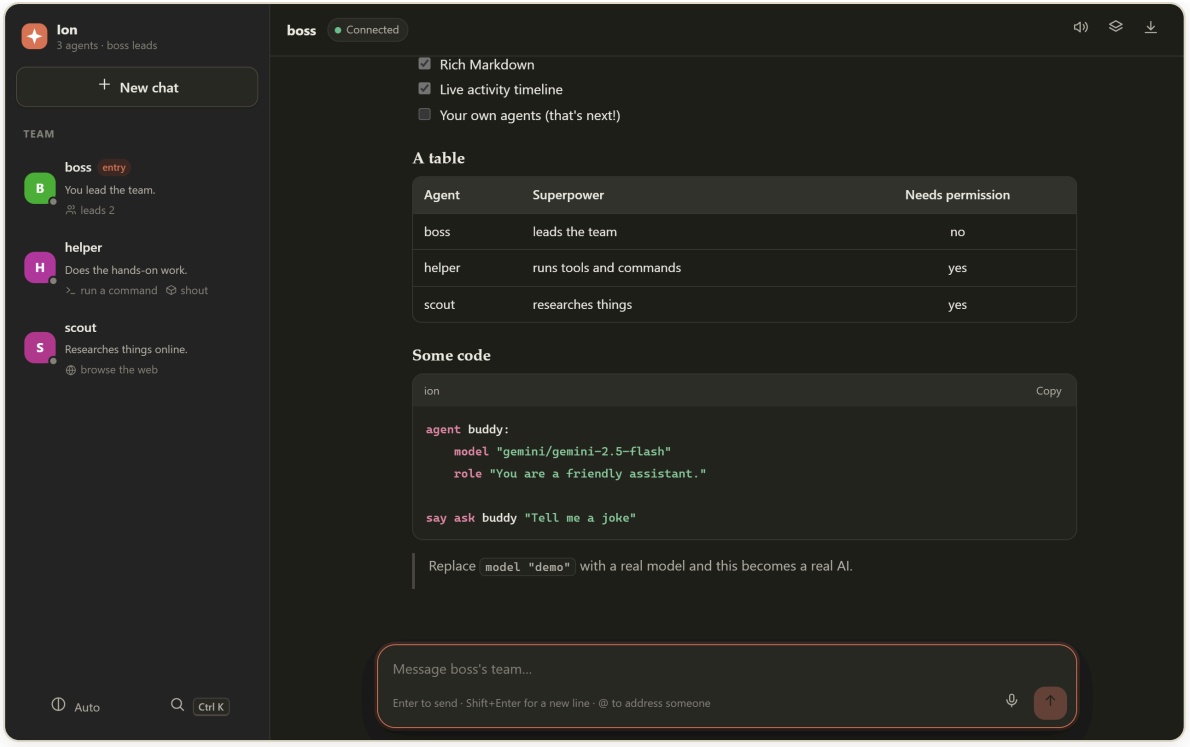

The command menu

## 16.7 Keyboard shortcuts

| Keys | Does |
|---|---|
| Enter | Send |
| Shift+Enter | New line |
| `@` | Address an agent |
| `/` | Jump to the message box |
| Ctrl+K | Command menu |
| Ctrl+Shift+O | New chat |
| Y / N | Allow / deny a permission request |
| Esc | Close menus, stop dictation, close the side panel |

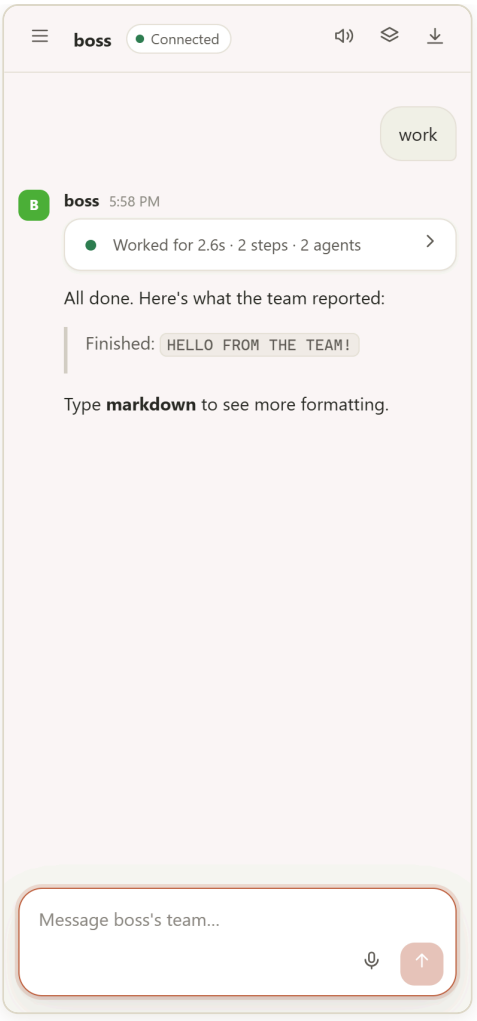
## 16.8 Themes, phone, voice

- **Themes.** Light, dark, or follow your computer. Use the button at the bottom of the side panel.




Dark theme

- **Phone and narrow windows.** The side panel becomes a drawer behind a menu button.




On a phone

- **Voice.** If an agent has `input text, voice`, a microphone button appears: speak and your words are typed live, then sent. The speaker button in the title bar reads replies aloud (turn it on and it remembers). Speech uses your browser's built-in features, which work best in Chrome and Edge, and the browser asks for microphone permission.

## 16.9 History, export and starting over

- Your conversation is saved in your browser, so it is still there after a refresh. The agents themselves start fresh (their memory lives in the running program), and the page says so with a divider.
- **Export** (download icon) saves the chat as a Markdown file.
- **New chat** (or Ctrl+Shift+O) clears the screen and gives the agents a clean slate.

## 16.10 Previewing without any setup

```text
ion preview
```

Shows the chat page with a built-in demo team (no key needed). Type `work`, `permission`, `research` or `markdown` to see each feature. `ion preview myfile.ion` previews your own agents: if your file has a `serve` line it is used, otherwise the last team (or the first agent) is served.

## 16.11 Limits

- Replies appear when complete, not word by word. The live activity card shows progress instead.
- There is no stop button yet, because running work cannot be cancelled mid-flight. Extra messages queue.
- The page is for one person on your own computer. It has no accounts or login.

CHAPTER 17

# Splitting a program across files

Big programs are easier to read as several files. `use` runs another file first, and everything it defines (functions, agents, tools, teams, variables) becomes available:

ION

```
use "agents.ion"
use "helpers.ion"

say ask researcher "hello"
```

Details:

- Paths are relative to the file that contains the `use`.
- A file is run **only once**, even if several files use it.
- All files share the same names. A file that defines `agent researcher` makes `researcher` usable everywhere.
- If an error happens inside a used file, the message names that file and line.
- `ion check` is more lenient when a file uses others, because names might come from them.

A good layout:

TEXT

```
my_project/
    agents.ion      # agents and tools only
    helpers.ion     # plain functions
    main.ion        # use "agents.ion", use "helpers.ion", then the actual program
    .env            # keys
```

CHAPTER 18

# Command-line reference

If you did not run `npm link`, replace `ion` with `npx tsx src/cli/index.ts`.

## 18.1 `ion run`

```
TEXT

ion run <file.ion> [--yes] [--headless] [--port N]
```

| Option | Meaning |
|---|---|
| `--yes` | Approve every permission prompt automatically. **Only for programs you trust.** |
| `--headless` | With `serve`: API only, no web page, no browser |
| `--port N` | Overrides the port in a `serve` statement |

Runs the program. If it fails, the error is printed and the exit status is 1.

## 18.2 `ion check`

```
TEXT

ion check <file.ion>
```

Reports mistakes without running anything: unknown agents, tools or models, teams with a missing lead, duplicate names, misspelled settings, and `ask` targets that do not exist. Prints `file: ok` when clean.

## 18.3 `ion repl`

```
TEXT

ion repl [--yes]
```

An interactive prompt. Type statements one at a time. End a line with `:` to start a block and finish with an empty line. Type `exit` to leave. Variables, functions and agents you define stay for the whole session.

## 18.4 `ion preview`

```
TEXT

ion preview [file.ion] [--port N] [--no-open] [--yes]
```

Opens the chat page. With no file, a built-in demo. With a file, your own agents. `--no-open` starts the server without opening a browser.

## 18.5 The `.env` file

Before running, Ion looks for a `.env` file in the folder of your script (or, failing that, the current folder) and loads `NAME=value` lines into the environment. Variables already set in your terminal are never overwritten.

## 18.6 Old-style files

Files whose first line is `ion init` use the original brace syntax and keep working with `ion run` and `ion check` (Appendix E).

CHAPTER 19

# Cookbook: complete programs

Ready-to-adapt programs. Recipes that use a real model say so. Change the `model` line to the provider you have a key for.

## 19.1 A study buddy

A chat loop you control yourself, so you can add your own logic around each message:

ION

```
agent tutor:
    model "gemini/gemini-2.5-flash"
    role "You are a patient tutor. Explain simply, with one short example."

say "Ask me anything. Type quit to stop."
repeat forever:
    set question to ask user "You:"
    if question is nothing or question is "quit":
        stop
    say "Tutor: " + ask tutor question
```

> **TIP**
> For a plain chat you can just write `chat with tutor`. Use a loop like this when you want to do something to each message, such as logging it or checking for special commands.

## 19.2 Summarize a file

ION

```
agent reader:
    model "gemini/gemini-2.5-flash"
    role "You summarize documents in three bullet points."
    can access files in "./documents"

say ask reader "Read notes.txt and summarize it."
```

The agent reads `./documents/notes.txt` by itself, asking your permission the first time. It cannot read anything outside `./documents`.

## 19.3 Turn messy text into data

Ask for JSON, then use the result like any map:

```ion
agent extractor:
    model "openai/gpt-4o-mini"
    role "You extract facts. Reply with JSON only."
    forgets between asks

set raw to ask extractor 'From "Ada Lovelace wrote the first program in 1843", reply as
{"name": "...", "year": 0}'
set data to json(raw)
say "{data.name} did it in {data.year}"
```

`json()` copes with code fences and extra chatter around the data.

## 19.4 An offline customer-support bot with a tool

Runs with the mock AI so you can see the whole flow for free. Swap `"mock"` for a real model and let the AI decide when to look up an order:

```ion
set orders to {
    A100: "shipped on Monday",
    A101: "out for delivery",
    A102: "being packed"
}

tool order_status(order_id):
    describe "Looks up the status of an order by its id, like A100"
    set found to orders[order_id]
    if found is nothing:
        give back "No order with id " + order_id
    give back found

agent support:
    model "mock"
    role "You are a friendly support agent. Use order_status to answer questions about orders."
```

```
    can use order_status

say ask support 'CALL order_status {"order_id": "A101"}'
say ask support 'CALL order_status {"order_id": "Z999"}'
```

OUTPUT

```
Done: out for delivery
Done: No order with id Z999
```

## 19.5 Compare several AIs

ION

```
agent gemini_a:
    model "gemini/gemini-2.5-flash"
agent openai_a:
    model "openai/gpt-4o-mini"
agent claude_a:
    model "anthropic/claude-sonnet-5"

set names to ["Gemini", "OpenAI", "Claude"]
set answers to ask_all([gemini_a, openai_a, claude_a], "In one sentence: what is a black hole?")
for each i in range(3):
    say "{names[i]}: {answers[i]}"
```

All three are asked **simultaneously**.

## 19.6 A research team with a web page

ION

```
agent lead:
    model "gemini/gemini-2.5-flash"
    role "You coordinate. Send research to the researcher, then ask the writer to save a short report."
    input text, voice

agent researcher:
    model "gemini/gemini-2.5-flash"
    role "Find facts on the web. Always include the URLs you used."
    can browse the web
```

```
agent writer:
    model "gemini/gemini-2.5-flash"
    role "Turn research notes into a clear one-page report and save it as a file."
    can access files in "./reports"

team crew:
    lead lead
    members researcher, writer

serve crew
```

Run it, open the page, and ask: "*Research the history of the bicycle and save a report.*" Watch the timeline show the hand-offs, then approve the web and file permissions as they appear.

## 19.7 Retry when a service is busy

ION

```
set answer to nothing
set attempts to 0
while answer is nothing and attempts is less than 3:
    set attempts to attempts + 1
    try:
        set answer to ask helper "Summarize the news"
    on error:
        say "Attempt {attempts} failed: {error}"
        sleep(2)
if answer is nothing:
    say "Gave up after {attempts} tries"
otherwise:
    say answer
```

## 19.8 Remember things between runs

Agents forget when the program ends. To keep something, write it to a file and read it back:

ION

```
set notes to ""
try:
    set notes to read_file("notes.txt")
on error:
    say "(first run, no notes yet)"
```

```
agent buddy:
    model "gemini/gemini-2.5-flash"
    role "You are a helpful assistant. What you know about the user so far: {notes}"

set name to ask user "Tell me something about yourself:"
write_file("notes.txt", notes + "\n" + name)
say ask buddy "Reply to this in one friendly sentence: " + name
```

## 19.9 Process a list of items

ION

```
agent summarizer:
    model "openai/gpt-4o-mini"
    role "Summarize the text in one short sentence."
    forgets between asks

set articles to [
    "Ion is a language for building AI agents in plain English.",
    "Agents can browse the web, read files, and work together in teams."
]

set summaries to []
for each article in articles:
    try:
        add ask summarizer article to summaries
    on error:
        add "(failed: {error})" to summaries

write_file("digest.txt", join(summaries, "\n"))
say "Saved {length(summaries)} summaries to digest.txt"
```

## 19.10 Using an agent inside your own function

ION

```
agent translator:
    model "mock"
    forgets between asks

to translate(text, language):
```

```
    give back ask translator "Translate to {language}: {text}"

say translate("good morning", "French")
say translate("thank you", "Spanish")
```

OUTPUT

```
mock reply: Translate to French: good morning
mock reply: Translate to Spanish: thank you
```

Wrapping asks in functions keeps your program tidy and lets you reuse them.

CHAPTER 20

# Debugging and FAQ

## 20.1 A checklist when something is wrong

1. **Read the error.** The line number and hint usually tell you everything.
2. Run `ion check`. It catches misspelled names and missing pieces.
3. Run with `model "mock"`. If the logic works with the mock, the problem is with the real AI or your key, not your program.
4. `say` **things** to see what values really are. `say type_of(x)` is a great way to find "number vs text" confusion.
5. Use `ion repl` to test a small piece in isolation.

## 20.2 Common mistakes

| Symptom | Cause and fix |
|---|---|
| `I expected ':'` | A line that starts a block (`if`, `for each`, `agent`...) is missing its colon |
| `This line is indented differently...` | Inconsistent spaces inside a block |
| `I don't know anything called 'x'` | Typo, or you used a variable before `set`-ting it. The hint suggests the closest name |
| `Ion uses 'to' instead of '='` | Write `set x to 5`, not `set x = 5` |
| `I can't subtract text and number` | One value is text. Convert with `number(...)` |
| `5` and `"5"` are not equal | One is a number, one is text |
| `I expected 'times'` | `repeat` needs `repeat 3 times:` |
| A function changes a variable unexpectedly | `set` updates an existing outer variable (Section 8.4). Rename it |
| My JSON text prints a stray `{` | A `{` followed by a letter starts an interpolation. Write `\{` |
| `positions start at 0` error | Lists count from 0. The last item is `-1` |
| Agent never uses my tool | Improve the tool's `describe` line and the agent's `role`. Check the agent has `can use` |
| `Error: ... was denied` | You answered `n` to a permission prompt (or ran over HTTP without `--yes`) |
| `EADDRINUSE` when serving | The port is busy. Use `serve x on port 5000` |
| Web browsing says a browser is missing | Run `npx playwright install chromium` once |

## 20.3 FAQ

**Is Ion free?** Ion is. Real AI models charge you per use (the `mock` and `demo` models are free).

**Where does my data go?** Only to the AI provider you choose for each agent, and to any web pages your agents or functions fetch. Nothing goes to Ion's authors, because there is no Ion server.

**Can agents remember across runs?** Not automatically. Save to a file and read it back (Recipe 19.8).

**Can I use a model that runs on my own computer?** Not in the newer syntax yet. The older `ion init` syntax has a `custom` provider with an endpoint.

**Can two agents talk to each other?** Yes: `can ask`, teams, or a script that passes replies along (Chapter 14).

**Can I run Ion programs on a schedule?** Yes. `ion run file.ion` is just a command, so Windows Task Scheduler, cron, or CI can run it. Remember to supply keys through environment variables, and be careful with `--yes`.

**How do I stop a running program?** Press Ctrl+C.

**What does "tool-calling turns" in an error mean?** An agent used tools more than 25 times for a single message, which usually means it is stuck in a loop. Check your tool's `describe` and the agent's `role`.

**Is `can run commands` safe?** Only as safe as the agent and your attention. Read each command before you allow it. Give this ability to as few agents as possible.

## 20.4 Where to go from here

You now know the whole language. To grow:

1. Re-do the exercises you skipped (answers in Appendix F).
2. Build something small and useful for yourself: a file summarizer, a daily digest, a study buddy.
3. Add a tool that connects to a service you use.
4. Turn it into a team, and `serve` it.
5. Share your program (with the key moved to `.env`!).

Happy building.

APPENDIX A

# Cheat sheet

## Statements

| Write | Does |
|---|---|
| `say value` | Print |
| `set name to value` | Create or change a variable |
| `set list[0] to v`, `set map.key to v` | Change an item or field |
| `add value to list` | Append to a list |
| `remove value from list_or_map` | Remove an item or key |
| `if cond:` / `otherwise if cond:` / `otherwise:` | Decide |
| `repeat 3 times:` | Repeat a fixed number of times |
| `repeat forever:` | Repeat until `stop` |
| `while cond:` | Repeat while true |
| `for each x in things:` | Go through a list, text or map |
| `stop` / `skip` | Leave the loop / next round |
| `to name(a, b):` … `give back value` | Define a function |
| `try:` … `on error:` / `on error as e:` | Catch failures |
| `use "file.ion"` | Include another file |
| `describe "text"` | Document a function or tool |

## Agents, tools and teams

| Write | Does |
|---|---|
| `agent name:` | Define an agent (settings indented below) |
| `model "provider/name"` | Which AI (required) |
| `role "text"` | Instructions and personality |
| `key "..."` / `key from env "NAME"` | API key |
| `can access files in "folder"` | Fenced file access |
| `can browse the web` | Fetch pages, open a browser |
| `can run commands` | Run commands (not fenced!) |
| `can read gmail with token from env "X"` | Gmail |
| `can use tool_a, tool_b` | Use your tools |
| `can ask agent_a, team_b` | Delegate to others |
| `remembers` / `forgets between asks` | Memory |
| `input text, voice` / `output text, voice` | Chat page modes |
| `tool name(a):` + `describe "..."` | Define a tool |
| `team name:` + `lead x` + `members a, b` | Define a team |
| `ask who "message"` | Ask; gives back text |
| `ask user "question"` | Ask the person typing |
| `chat with who` | Terminal chat |
| `serve who [on port N]` | Chat web page and API |

## Expressions

```text
"text {inside}"   'single quotes'   """multi-line"""
42   3.5   -1   yes   no   nothing
[1, 2, 3]   {name: "Ada"}   list[0]   map.name   map["name"]
+ - * / %   plus minus times "divided by" mod
is   "is not"   "is greater than"   "is less than"   "is at least"   "is at most"
contains   "does not contain"   "is in"   "starts with"   "ends with"
and   or   not
```

# Commands

```text
ion run file.ion [--yes] [--headless] [--port N]
ion check file.ion
ion repl
ion preview [file.ion] [--port N] [--no-open]
```

APPENDIX B

# Reserved words

These words have a special meaning and cannot be used as names:

`set` `say` `add` `remove` `if` `otherwise` `else` `repeat` `for` `while` `stop` `skip` `to` `give` `try` `use` `agent` `team` `tool` `chat` `serve` `describe` `ask` `user` `and` `or` `not` `is` `in` `true` `false` `yes` `no` `nothing` `contains`

Other words (`each`, `back`, `on`, `error`, `as`, `from`, `times`, `forever`, `lead`, `members`, `model`, `role`, `key`, `can`, `input`, `output`) have meaning only in particular places, and you can still use most of them as names, though it is clearer not to.

APPENDIX C

# Error message glossary

| Message | What it means and what to do |
|---|---|
| `I don't know anything called 'x'` | A name was used that was never created. Check spelling and order |
| `I expected ':' here` | A block-starting line needs a colon at the end |
| `I expected an indented block` | After a `:` the next line must be indented |
| `This line is indented differently` | Misaligned spaces inside a block |
| `I don't know how to start a line with 'x'` | A misspelled statement word. See the hint |
| `Ion uses 'to' instead of '='` | Write `set x to ...` |
| `I can't add/subtract/multiply/divide X and Y` | Wrong types. Convert with `number()` or `text()` |
| `I can't compare X with Y` | `<` and `>` need two numbers or two texts |
| `You can't divide by zero.` | Check the divisor |
| `This list has N items, so there is no position P` | Positions run from 0 to N-1 (or -1 for the last) |
| `'x' isn't a function` | You used brackets on something that is not a function |
| `f(a, b) needs 2 inputs but got 1` | Wrong number of inputs |
| `'stop' and 'skip' only work inside a loop` | Move them into a loop |
| `'x' keeps calling itself too deeply` | A recursive function never stops |
| `Agent 'a' needs a model` | Add `model "provider/name"` |
| `I don't know the AI provider 'x'` | Use gemini, openai, anthropic, mock or demo |
| `needs the environment variable X, but it isn't set` | Provide the key (Chapter 12) |
| `Asking 'a' failed: ...` | The AI service returned an error. The text after it explains |
| `There is already an agent, team or tool called 'x'` | Names must be unique |
| `Agents are asking each other too many levels deep` | A delegation loop. Redesign who asks whom |
| `exceeded the maximum of 25 tool-calling turns` | An agent is stuck repeating tool use |
| `I can only add to a list` | `add` needs a list on the right of `to` |
| `can use 'x', but no such tool exists` | The tool name is misspelled or not defined |

APPENDIX D

# Grammar summary

A loose map of how the language is put together.

TEXT

```
program      = statement*
block        = ":" NEWLINE INDENT statement+ DEDENT

statement    = "say" expr
             | "set" target "to" expr
             | "add" expr "to" target
             | "remove" expr "from" target
             | "if" expr block ( "otherwise" "if" expr block )* [ "otherwise" block ]
             | "repeat" expr "times" block
             | "repeat" "forever" block
             | "while" expr block
             | "for" "each" name "in" expr block
             | "stop" | "skip"
             | "to" name "(" names ")" block
             | "give" "back" [ expr ]
             | "try" block "on" "error" [ "as" name ] block
             | "use" text
             | "describe" text
             | "agent" name ":" settings
             | "team" name ":" ( "lead" name | "members" names )+
             | "tool" name "(" names ")" block
             | "chat" [ "with" ] name
             | "serve" name [ "on" [ "port" ] expr ]
             | expr

expr         = or-expr
or-expr      = and-expr ( "or" and-expr )*
and-expr     = not-expr ( "and" not-expr )*
not-expr     = "not" not-expr | comparison
comparison   = sum ( comparator sum )*
sum          = product ( ("+" | "-" | "plus" | "minus") product )*
product      = unary ( ("*" | "/" | "%" | "times" | "divided by" | "mod") unary )*
unary        = "-" unary | postfix
```

```
postfix      = primary ( "(" args ")" | "[" expr "]" | "." name )*
primary      = number | text | yes | no | nothing | name
             | "[" exprs "]" | "{" fields "}" | "(" expr ")"
             | "ask" ( name | "user" ) sum
```

APPENDIX E

# The older `ion init` syntax

The first version of Ion used braces and `==`. Those files still run:

```
TEXT

ion init
type == team

agentdef orchestrator{
ai-provider == gemini
ai-apikey == env:GEMINI_API_KEY
ai-model == gemini/gemini-2.5-flash
grant orchestration-powers
input == text, voice
output == text}

agentdef researcher{
ai-provider == openai
ai-apikey == env:OPENAI_API_KEY
grant web-browsing
grant file-access 'C:/Users/me'}
```

A file is treated as the older syntax when its first line is `ion init`. Both syntaxes share the same engine, so the chat page, permissions and providers behave the same. New programs should use the syntax in this book. The older `custom` provider (your own endpoint) is available only there.

APPENDIX F

# Answers to the exercises

Try each exercise yourself first! Your answer may differ and still be right.

## Chapter 1

**Exercise 1.** `say "My name is Ada"`
**Exercise 2.** The lines print in the order they are written, top to bottom.
**Exercise 3.** In the REPL, `say "hi"` prints `hi`, and `exit` leaves.

## Chapter 2

**Exercise 1.**

ION

```
set name to "Ada"
set number to 7
say "{name}'s favorite number is {number}"
```

OUTPUT

```
Ada's favorite number is 7
```

**Exercise 2.** Division gives `3.5`, and `%` gives the remainder:

ION

```
say 7 / 2
say 7 % 2
```

OUTPUT

```
3.5
1
```

**Exercise 3.** The `say` line was indented, but nothing above it asks for a block. Remove the spaces:

ION

```
set count to 1
say count
```

OUTPUT

```
1
```

# Chapter 3

### Exercise 1.

ION

```
set name to "Ada"
set age to 36
say "Hi, I'm {name} and I'm {age}."
```

OUTPUT

```
Hi, I'm Ada and I'm 36.
```

### Exercise 2.

ION

```
say """
    Roses are red,
    Violets are blue,
    Ion is easy.
    """
```

OUTPUT

```
Roses are red,
Violets are blue,
Ion is easy.
```

**Exercise 3.** `"5" + 3` is the text `"53"`, because adding to text joins them. `5 + 3` is the number `8`.

ION

```
say "5" + 3
say 5 + 3
```

OUTPUT

```
53
8
```

**Exercise 4.** `0`, `[]` and `nothing` are false. `"0"` (non-empty text) and `[0]` (non-empty list) are true.

ION

```
for each v in [0, "0", [], [0], nothing]:
    if v:
        say "{v} is true"
    otherwise:
```

```
        say "{v} is false"
```

OUTPUT

```
0 is false
0 is true
[] is false
[0] is true
nothing is false
```

# Chapter 4

### Exercise 1.

ION

```
say (8 + 12 + 22) / 3
```

OUTPUT

```
14
```

### Exercise 2.

ION

```
set age to 15
say age is at least 13 and age is at most 19
```

OUTPUT

```
yes
```

### Exercise 3.

ION

```
set word to "Level"
say lower(word) is "level"
```

OUTPUT

```
yes
```

**Exercise 4.** It prints `default`. `0` is false, so `or` gives back its second input.

ION

```
say 0 or "default"
```

OUTPUT

```
default
```

# Chapter 5

### Exercise 1.

ION

```ion
set n to 7
if n % 2 is 0:
    say "even"
otherwise:
    say "odd"
```

OUTPUT

```
odd
```

### Exercise 2.

ION

```ion
set score to 72
if score is at least 90:
    say "A"
otherwise if score is at least 80:
    say "B"
otherwise if score is at least 70:
    say "C"
otherwise if score is at least 60:
    say "D"
otherwise:
    say "F"
```

OUTPUT

```
C
```

### Exercise 3.

ION

```ion
set hour to 15
if hour is less than 12:
    say "Good morning"
otherwise if hour is less than 18:
```

```
    say "Good afternoon"
otherwise:
    say "Good evening"
```

OUTPUT

```
Good afternoon
```

# Chapter 6

### Exercise 1.

ION

```
set line to ""
for each i in range(1, 11):
    set line to line + i + " "
say trim(line)
```

OUTPUT

```
1 2 3 4 5 6 7 8 9 10
```

### Exercise 2.

ION

```
set result to []
for each n in range(1, 31):
    if n % 3 is 0:
        add n to result
say result
```

OUTPUT

```
[3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
```

### Exercise 3.

ION

```
set n to 1
while n is at most 1000:
    set n to n * 2
say n
```

OUTPUT

```
1024
```

### Exercise 4.

```ion
set evens to []
for each i in range(1, 6):
    add i * 2 to evens
say evens
```

```
OUTPUT
[2, 4, 6, 8, 10]
```

## Chapter 7

### Exercise 1.

```ion
set nums to [4, 8, 15, 16, 23]
say sum(nums)
say max(nums)
```

```
OUTPUT
66
23
```

### Exercise 2.

```ion
set book to {title: "Dune", author: "Frank Herbert", year: 1965}
say "{book.title} was written by {book.author} in {book.year}."
```

```
OUTPUT
Dune was written by Frank Herbert in 1965.
```

### Exercise 3.

```ion
set words to ["pear", "fig", "apple"]
say sort(words)

to long_word(w):
```

```
    give back length(w) is greater than 3

say filter(words, long_word)
```

OUTPUT

```
["apple", "fig", "pear"]
["pear", "apple"]
```

### Exercise 4.

ION

```
set text_form to to_json({a: 1, b: 2})
say text_form
say json(text_form).b
```

OUTPUT

```
{"a":1,"b":2}
2
```

# Chapter 8

### Exercise 1.

ION

```
to double(n):
    give back n * 2
say double(21)
```

OUTPUT

```
42
```

### Exercise 2.

ION

```
to biggest(numbers):
    set best to numbers[0]
    for each n in numbers:
        if n is greater than best:
            set best to n
```

```
    give back best

say biggest([3, 9, 2])
say max([3, 9, 2])
```

OUTPUT

```
9
9
```

**Exercise 3.**

ION

```
to is_even(n):
    give back n % 2 is 0

say filter(range(10), is_even)
```

OUTPUT

```
[0, 2, 4, 6, 8]
```

**Exercise 4.**

ION

```
to repeat_text(text, count):
    set out to ""
    repeat count times:
        set out to out + text
    give back out

say repeat_text("ab", 3)
```

OUTPUT

```
ababab
```

# Chapter 9

**Exercise 1.** Any typo works. For example `say totl` shows `I don't know anything called 'totl'` with the hint `Did you mean 'total'?`.

**Exercise 2.**

ION

```
try:
    say number("12abc")
on error:
    say "That is not a number, sorry."
```

OUTPUT

```
That is not a number, sorry.
```

**Exercise 3.**

ION

```
to safe_divide(a, b):
    if b is 0:
        give back nothing
    give back a / b

say safe_divide(10, 4)
say safe_divide(1, 0)
```

OUTPUT

```
2.5
nothing
```

# Chapter 10

**Exercise 1.**

ION

```
say join(map(split("red green blue"), upper), ", ")
```

OUTPUT

```
RED, GREEN, BLUE
```

**Exercise 2.** (The result is random.)

ION

```
say random(1, 6) + random(1, 6)
```

**Exercise 3.**

ION

```
say "Today is " + today()
```

# Chapter 11

**Exercises 1 and 2.**

ION

```
agent a:
    model "mock"
agent b:
    model "mock"

say ask a "What is 2+2?"
say ask b "What is 2+2?"
```

OUTPUT

```
mock reply: What is 2+2?
mock reply: What is 2+2?
```

**Exercise 3.**

ION

```
agent poet:
    model "gemini/gemini-2.5-flash"
    role "You answer every question in a short rhyming couplet."
```

**Exercise 4.** A translator should **forget**. Each translation is independent, and old messages would only add cost.

# Chapter 12

**Exercise 1.** A fake key is rejected by the provider, so you see `Asking 'a' failed:` followed by a 400-style message about the key. That is the correct outcome.
**Exercise 2.** `key from env "MY_OWN_VARIABLE"` inside the agent.
**Exercise 3.** An inline key travels with the file. Anyone you send it to gets your key (and your bill). A `.env` file stays on your computer.

# Chapter 13

**Exercise 1.**

ION

```
tool word_count(text):
    describe "Counts the words in some text"
    give back length(split(text))

say word_count("one two three")
```

OUTPUT

```
3
```

**Exercise 2.**

ION

```
tool word_count(text):
    describe "Counts the words in some text"
    give back length(split(text))

agent counter:
    model "mock"
    can use word_count

say ask counter 'CALL word_count {"text": "a b c d"}'
```

OUTPUT

```
Done: 4
```

**Exercise 3.**

ION

```
tool tip(amount, percent):
    describe "Calculates a tip for a bill amount and percentage"
    give back amount * percent / 100

say tip(80, 15)
```

OUTPUT

```
12
```

**Exercise 4.** For example: "*Looks up the price of a product by name. Input: the product name. Returns the price in dollars, or says it was not found.*"

# Chapter 14

### Exercise 1.

```ion
agent boss:
    model "mock"
agent a:
    model "mock"
agent b:
    model "mock"

team squad:
    lead boss
    members a, b

say ask squad 'CALL delegate_to_a {"task": "first"}'
say ask squad 'CALL delegate_to_b {"task": "second"}'
```

```
Done: mock reply: first
Done: mock reply: second
```

### Exercise 2.

```ion
agent first:
    model "mock"
agent second:
    model "mock"

set a to ask first "hello"
say ask second "Please improve: " + a
```

```
mock reply: Please improve: mock reply: hello
```

### Exercise 3.

```ion

```

```
agent p:
    model "mock"
agent q:
    model "mock"
agent r:
    model "mock"

for each reply in ask_all([p, q, r], "ping"):
    say reply
```

OUTPUT

```
mock reply: ping
mock reply: ping
mock reply: ping
```